09 API Key & Secret Leak Detector

09-git-secret-scanner

Scans every commit of a git repo's history for leaked keys (regex + entropy), dedupes by fingerprint, reports only redacted values, and blocks new leaks with a pre-commit hook.

Verified here: 60 passed; 0 failed; 0 skipped

Measured results

- Found 11/11 planted fake secrets in an 8-commit demo repo, 0 false positives, 0/8 benign look-alikes flagged
- 6 of them caught only in history (deleted or rotated), marked removed_from_HEAD_still_in_history
- 12 raw occurrences deduped to 11 unique secrets; 3/3 allowlisted correctly (inline, path, fingerprint)
- Pre-commit hook: clean commit exit 0, commit with a fake Stripe key exit 1 (blocked)
- 0 raw secret values in JSON/HTML/console output (test-enforced)
- Real repos: 17,777 commits of requests/boto3/itsdangerous/sampleproject in 84.4 s (~211 commits/s), 7 findings, all docs example keys or TLS test fixtures

Data provenance and caveats

- Generated demo git repo (src/sample_repo.py, deterministic, 8 commits) with only FAKE / vendor-documentation example secrets; it is created in a temp dir and no .git folder is shipped
- Real public repos cloned for measurement only: https://github.com/pypa/sampleproject, https://github.com/pallets/itsdangerous, https://github.com/psf/requests, https://github.com/boto/boto3 (HEADs recorded in outputs/public_repo_scans.json)
- Demo recall/false-positive numbers are on the project's own generated ground truth, not a real-world benchmark
- Real-repo findings were triaged by file path/context only (docs example keys, TLS test fixtures); no key was tested against any provider and no precision-vs-real-leaks number is claimed
- No live validation of secrets: 'active' means present in HEAD, not valid at the provider
- Commit messages, binaries, stashes and merge-only conflict resolutions (patch mode) are not scanned
- Fingerprints are unsalted SHA-256 (short secrets could be brute-forced from a report)

Project contents

README.md, requirements.txt, cli.py, app.py, src/__init__.py, src/config.py, src/entropy.py, src/hook.py, src/redact.py, src/report.py, src/rules.py, src/sample_repo.py, src/scanner.py, tests/conftest.py, tests/test_app_selfscan.py, tests/test_history.py, tests/test_hook_cli.py, tests/test_rules.py, .gitignore, config/secretscan.example.yml, pytest.ini, scripts/run_demo.py, scripts/scan_public_repos.py

secretscan: API Key & Secret Leak Detection Engine for Git History

Scans **every commit** of a git repository, including secrets that were later deleted or "rotated", with provider regex rules plus Shannon-entropy detection. Findings are deduplicated by fingerprint, and reports only ever hold **redacted** values. Ships with a blocking pre-commit hook.

Problem

Deleting a leaked API key in a new commit does not remove it: the key stays in git history, and anyone who clones the repo can read it. Leaked cloud keys are one of the most common causes of cloud-account compromise. Teams need two things:

1. A way to audit a repo's **whole history** to find out what has ever leaked, and whether it is still in the current code.
2. A guard that **stops new secrets** before they are committed.

How it works / Architecture

```
              +--------------------+
 git repo ?????? |  GitPython walker  |  all refs (--all), oldest ? newest
              +---------+----------+
        patch mode         |          blobs mode
  diff(first parent, c) -U0?   every unique blob in every tree (scanned once per blob SHA)
        only "+" lines     ?
              +--------------------+     +--------------------------+
              |   line scanner     |?????| rules.py                 |
              |  provider regexes  |     |  AWS / GitHub / Slack /  |
              |  + generic entropy |     |  Stripe / Google / PEM   |
              +---------+----------+     |  generic assignment +    |
                        ?                |  Shannon entropy gate    |
                        ?                +--------------------------+
              +--------------------+
              | allowlist          |  inline comment · path glob · fingerprint · value
regex
              +---------+----------+
                        ?
              +--------------------+     fingerprint =
sha256("secretscan:v1:"+value)[:24]
              | dedupe + HEAD check|     status = active_in_HEAD |
removed_from_HEAD_still_in_history
              +---------+----------+
                        ?
           JSON report · HTML report · console · Streamlit dashboard   (redacted only)

 git commit ??? .git/hooks/pre-commit ??? cli.py precommit ??? scan `git diff --cached`
??? exit 1 = blocked
```

- **History scan (`patch` mode, the default):** for each commit, the scanner diffs against the first parent (or git's empty tree for a root commit) with `-U0 -M` and scans only the added lines. Each secret is attributed to the commit that introduced it, with the new-file line number taken from the hunk headers. Renames (`-M`) are not reported again.
- **`blobs` mode:** scans every unique file version that ever existed, as an independent cross-check.
- **Rules:** provider shapes are AWS key IDs (`AKIA/ASIA/...` + 16), AWS secret keys (40 base64 characters, and only next to `aws?secret` context), GitHub `ghp_/gho_/ghu_/ghs_/ghr_` and fine-grained PATs, Slack `xox?-` tokens and webhooks, Stripe `sk_/rk_` keys, Google `AIza` keys, and PEM private key headers.

- **Generic rule:** a credential-like name (`password`, `secret`, `token`, `api_key`, ?) assigned a quoted literal of 16 or more characters is reported only if its **Shannon entropy is ? 3.5 bits/char**, it uses ? 3 character classes, and it isn't a placeholder (`${VAR}`, `<your-key>`, `changeme`, `xxxx?`).
- **Dedupe:** occurrences are grouped by fingerprint. The report lists each unique secret once, with every commit, file and line where it appears.
- **Allowlist:** `.secretscan.yml` in the scanned repo (or `--config`) supports `paths` (fnmatch globs), `fingerprints` (copied from a previous report) and `values_regex`. Adding `secretscan:allow` or `pragma: allowlist secret` to a line suppresses it inline. Suppressed matches are listed in their own report section with the reason.
- **Pre-commit hook:** `install-hook` writes `.git/hooks/pre-commit`, which runs `cli.py precommit`. That scans only the staged added lines (`git diff --cached`) and exits **1** if anything is found, so git aborts the commit.

Security design decisions

- **No raw secrets at rest.** `Finding.secret` lives only in memory during a scan and is removed when results are serialised. Reports carry a fingerprint plus a redacted form. The redacted form keeps only a *documented public type prefix* (e.g. `AKIA`, `ghp_`, capped at 4 characters). Generic secrets such as passwords show **zero** characters: `********(27 chars)`. Tests assert that no planted value, and no 10-character slice of one, appears in the JSON, HTML or console output.
- **Fingerprints, not values, for allowlisting.** A reviewer can allowlist a known false positive without writing the value into a config file. Caveat: the fingerprint is an unsalted SHA-256, so a short, low-entropy secret could be brute-forced from it. Treat reports as sensitive anyway.
- **PEM keys fingerprint by header + path.** Every PEM header line is identical, so without this every private key in a repo would collapse into a single finding. (A run on `psf/requests` exposed this; it is fixed, and a test covers it.)
- **No live validation.** The tool never sends a candidate secret to AWS, GitHub or anyone else. That avoids leaking it further and avoids touching third-party systems. As a result, "active" means "present in the HEAD tree", not "still valid at the provider".
- **HTML is autoescaped.** File paths come from untrusted repos. A test checks that a path like `<script>?` is escaped.
- **Read-only scanning.** The scanner only runs `git diff` and reads objects. It never checks out, executes or modifies the scanned repo (covered by a test).
- **Demo data that cannot authenticate.** Every planted value is either a vendor's own published documentation example (AWS `?EXAMPLE` keys, Slack's `T00000000/B00000000/XXXX?` webhook) or contains `FAKE`/`EXAMPLE`/`TEST` plus runs of zeros. The private-key file body is the literal text `FAKE-EXAMPLE-NOT-A-REAL-KEY-0000`. The generator assembles these values **from fragments at runtime**. As a result, this project's own source contains no contiguous secret-shaped string: GitHub push protection won't trip when you push it, and the self-scan test stays clean. The demo repo is created in a temp directory or a gitignored folder, and no `.git` folder is shipped.
- **Bounded work.** Lines longer than 5,000 characters (minified bundles) are truncated before regexes run, binary blobs are skipped, and every regex is linear (no nested quantifiers).

Project structure

```
09-git-secret-scanner/
??? cli.py                  # generate-sample | scan | precommit | install-hook | uninstall-hook
??? app.py                  # Streamlit dashboard (offline, reads outputs/report.json)
??? src/
?   ??? rules.py            # provider regex rules + generic assignment rule + placeholder filter
?   ??? entropy.py          # Shannon entropy, character classes
?   ??? scanner.py          # line scanner, unified-diff parser, history
```

```
(patch/blobs), staged scan, dedupe
?   ??? config.py              # YAML config + allowlist + inline suppression
?   ??? redact.py              # fingerprint + redaction
?   ??? report.py              # JSON / HTML (Jinja2, autoescaped) / console output
?   ??? hook.py                # pre-commit hook install / uninstall (backs up foreign
hooks)
?   ??? sample_repo.py         # deterministic demo repo generator with FAKE secrets +
ground truth
??? scripts/
?   ??? run_demo.py            # regenerates everything in outputs/ (temp repo, deleted
afterwards)
?   ??? scan_public_repos.py    # real-repo false-positive / speed measurement (network)
??? config/secretscan.example.yml
??? tests/                     # 60 pytest tests
??? outputs/                   # report.json, report.html, report_blobs.json,
findings_by_rule.png,
                               # public_repo_scans.json, sample_output.txt,
pytest_output.txt, results.json
```

Setup & how to run

```bash
pip install -r requirements.txt           # needs git >= 2.28 on PATH

# 1. create the throwaway demo repo (8 commits, FAKE secrets planted, rotated and deleted)
python cli.py generate-sample sample_repo

# 2. scan its full history (patch mode), write redacted reports
python cli.py scan sample_repo --json outputs/report.json --html outputs/report.html
python cli.py scan sample_repo --mode blobs          # cross-check over every file version
python cli.py scan . --fail-on-findings              # CI usage: exit 1 if anything is
reported

# 3. protect a repo with the pre-commit hook
python cli.py install-hook /path/to/your/repo        # --force backs up an existing hook
git commit ...                                       # blocked with exit 1 if a secret is
staged
python cli.py uninstall-hook /path/to/your/repo

# tests, demo, dashboard
pytest -v
python scripts/run_demo.py
python scripts/scan_public_repos.py                  # optional, clones 4 public repos
streamlit run app.py
```

Results (measured on this run)

Test suite: 60 passed, 0 failed (`outputs/pytest_output.txt`).

Demo repo (8 commits, 14 planted fake secrets with ground truth in
`src/sample_repo.py`):

Metric	Result
Commits / added lines scanned (patch mode)	8 / 60
Raw occurrences ? unique secrets after fingerprint dedupe	12 ? 11 (expected 11)
Planted secrets that should be reported, found	11 / 11 (100% recall), 0 false
positives

Deleted or rotated secrets still caught from history	6 (marked `removed_from_HEAD_still_in_history`)
Secrets still in HEAD	5
Allowlisted correctly (inline, path glob, fingerprint)	3 / 3
Benign look-alikes (placeholders, `${VAR}`, `os.environ`, low entropy) reported	0 / 8
Blobs-mode cross-check	13 occurrences in 16 unique blobs ? same 11 unique secrets
Scan time, patch / blobs	21.6 ms / 12.3 ms
Pre-commit hook: clean commit / commit with fake Stripe key	exit 0 / exit 1 (blocked, commit count stayed at 1)
Raw planted values found in JSON/HTML/console output	0

Real public repositories (default config, full history, patch mode; clones deleted afterwards):

Repo (HEAD date)	Commits	Added lines	Raw ? unique findings	Time
pypa/sampleproject (2024-11-06)	197	1,772	0 ? 0	0.4 s
pallets/itsdangerous (2025-06-14)	678	25,626	0 ? 0	3.0 s
psf/requests (2026-10-26)	6,737	327,148	12 ? 4	21.6 s
boto/boto3 (2026-10-26)	10,165	560,582	9 ? 3	59.4 s

Total: **17,777 commits / 915,128 added lines in 84.4 s (? 211 commits/s)** on this machine.

What the 7 unique real-repo findings were (by file path and context, not by testing any key):
- **boto3:** the AWS documentation example access key (its fingerprint matches the AWS docs example planted in the demo repo, which shows fingerprints are stable across repos) and a second docs-style example key/secret pair in a since-deleted docs page (the value ends in `EXAMPLE`).
- **requests:** 4 PEM private keys under `tests/certs/`, which are TLS test fixtures committed on purpose.

All 7 are correct pattern matches, and none looks like an accidentally leaked production credential. Each is exactly what a reviewer would allowlist by path or fingerprint. Precision against "real leaks" can't be measured without labelled data, so no such number is claimed.

Sample output

```
$ python cli.py generate-sample demo_repo
created demo repo with 8 commits (all planted secrets are FAKE examples)
  e767ee6cc482  Initial skeleton
  2936f184f88d  Add AWS settings for S3 uploads
  4157ed52bdc9  Add Slack notifications and CI script
  6bc44ca07e06  Rotate AWS key
  cd502734a317  Move AWS credentials to environment variables
  208edc553b58  Add payments, maps, DB config, tests and scanner config
  6dc4a70a5d63  Add deploy key
  3a114f323d9f  Remove deploy key (oops)

$ python cli.py scan demo_repo --json outputs/report.json --html outputs/report.html
secretscan: patch scan of demo_repo: 8 commits, 60 lines, 12 raw occurrences -> 11 unique secrets (3 suppressed) in 21.6 ms
  [CRITICAL] aws-access-key-id                AKIA********(20 chars)   fp=350561d85b7c x2 first=2936f184f88d app/settings.py:2  removed_from_HEAD_still_in_history
  [CRITICAL] aws-secret-access-key            ********(40 chars)       fp=37b8c3b91aec x1 first=2936f184f88d app/settings.py:3  removed_from_HEAD_still_in_history
```

```
  [CRITICAL] github-token                  ghp_********(40 chars)   fp=7d16d413d6db x1
first=4157ed52bdc9 scripts/ci.sh:3  active_in_HEAD
  [CRITICAL] aws-access-key-id               AKIA********(20 chars)   fp=2c267e30c471 x1
first=6bc44ca07e06 app/settings.py:2  removed_from_HEAD_still_in_history
  [CRITICAL] aws-secret-access-key           ********(40 chars)       fp=d998a934ec0a x1
first=6bc44ca07e06 app/settings.py:3  removed_from_HEAD_still_in_history
  [CRITICAL] stripe-secret-key               sk_t********(39 chars)   fp=d080eebd20d6 x1
first=208edc553b58 app/payments.py:1  active_in_HEAD
  [CRITICAL] private-key-block               ----********(35 chars)   fp=8529a64e32b5 x1
first=6dc4a70a5d63 keys/deploy_key:1  removed_from_HEAD_still_in_history
  [HIGH    ] slack-token                   xoxb********(51 chars)   fp=0d6b029f76f3 x1
first=4157ed52bdc9 app/notify.py:2  removed_from_HEAD_still_in_history
  [HIGH    ] slack-webhook                 http********(77 chars)   fp=7edd89cfbcef x1
first=4157ed52bdc9 app/notify.py:1  active_in_HEAD
  [HIGH    ] google-api-key                AIza********(39 chars)   fp=c8b47645db14 x1
first=208edc553b58 web/maps.js:1  active_in_HEAD
  [MEDIUM  ] generic-high-entropy-assignment  ********(27 chars)       fp=f4a10f8ea933 x1
first=208edc553b58 app/db.py:1  active_in_HEAD

$ python cli.py scan demo_repo --mode blobs
secretscan: blobs scan of demo_repo: 8 commits, 66 lines, 13 raw occurrences -> 11 unique
secrets (3 suppressed) in 12.3 ms

$ python cli.py install-hook hook_repo
installed pre-commit hook at hook_repo/.git/hooks/pre-commit

$ git commit -m 'clean commit'   -> exit 0
[master (root-commit) f3c4f81] clean commit
 1 file changed, 1 insertion(+)
 create mode 100644 app.py

$ git commit -m 'add stripe key'   -> exit 1
secretscan: commit BLOCKED - likely secrets in staged changes:
  config.py:1  stripe-secret-key  sk_t********(39 chars)  fp=d080eebd20d6
Remove them (use env vars / a secret manager), or allowlist a false positive with an
inline
'secretscan:allow' comment or its fingerprint in .secretscan.yml. Bypass: git commit --no-
verify

self-check: raw planted secrets found in report.json/report.html/report_blobs.json/console
 output: 0
```

The HTML report is at `outputs/report.html` and the chart at
`outputs/findings_by_rule.png`.

Limitations & future work

- **Regexes find shapes, not validity.** There is no provider verification. A rotated key
that was revoked at the provider is still reported (that is intentional), and novel token
formats are missed.
- **The generic rule is a heuristic.** High-entropy strings without a credential-like name
 (e.g. a bare token in a URL) are missed, and random IDs assigned to `token = "?"` produce
 false positives. The 3.5 bits/char threshold is configurable.
- **Not scanned:** commit messages, notes, stashes, and binary files (including zipped or
encoded secrets). Patch mode follows first parents only, so changes introduced *only* by
merge-conflict resolution are missed (blobs mode covers them).
- Values split across lines or built by string concatenation are not detected.
- The pre-commit hook can be skipped with `git commit --no-verify`. Pair it with a server-
side or CI history scan (`scan --fail-on-findings`).

- Fingerprints are unsalted SHA-256 (see above). A per-org HMAC key would harden them.
- Future work: a rule for verifying GitHub token checksums, SARIF output for GitHub code scanning, parallel per-commit scanning, and an incremental "since last scanned commit" mode for CI.

Resume bullets

- Built a Python/GitPython **git-history secret scanner** that walks every commit's patch (and optionally every blob) with 9 provider rules plus a Shannon-entropy detector. On a seeded 8-commit test repo it found **11/11 planted fake secrets, including 6 deleted or rotated ones**, with 0 false positives.
- Designed **privacy-preserving reporting**: SHA-256 fingerprint deduplication (12 raw hits ? 11 unique) and prefix-only redaction. **0 raw secret values** appear in the JSON, HTML or console output, enforced by automated tests.
- Shipped a **blocking git pre-commit hook** with an installer, a YAML/fingerprint/inline allowlist, and a Streamlit dashboard, all covered by **60 passing pytest tests**, including real `git commit` runs that exit 1 on a staged key.
- Benchmarked on **17,777 commits of real open-source history** (requests, boto3, itsdangerous, sampleproject) at **? 211 commits/s**, and triaged all 7 findings (documentation example keys and TLS test fixtures).

API Key & Secret Leak Detector — ACTUAL SAMPLE OUTPUT

```
$ python cli.py generate-sample demo_repo
created demo repo with 8 commits (all planted secrets are FAKE examples)
  e767ee6cc482  Initial skeleton
  2936f184f88d  Add AWS settings for S3 uploads
  4157ed52bdc9  Add Slack notifications and CI script
  6bc44ca07e06  Rotate AWS key
  cd502734a317  Move AWS credentials to environment variables
  208edc553b58  Add payments, maps, DB config, tests and scanner config
  6dc4a70a5d63  Add deploy key
  3a114f323d9f  Remove deploy key (oops)

$ python cli.py scan demo_repo --json outputs/report.json --html outputs/report.html
secretscan: patch scan of demo_repo: 8 commits, 60 lines, 12 raw occurrences -> 11 unique secrets
(3 suppressed) in 21.6 ms
  [CRITICAL] aws-access-key-id                AKIA********(20 chars)   fp=350561d85b7c x2 first=29
36f184f88d app/settings.py:2  removed_from_HEAD_still_in_history
  [CRITICAL] aws-secret-access-key            ********(40 chars)       fp=37b8c3b91aec x1 first=29
36f184f88d app/settings.py:3  removed_from_HEAD_still_in_history
  [CRITICAL] github-token                     ghp_********(40 chars)   fp=7d16d413d6db x1 first=41
57ed52bdc9 scripts/ci.sh:3  active_in_HEAD
  [CRITICAL] aws-access-key-id                AKIA********(20 chars)   fp=2c267e30c471 x1 first=6b
c44ca07e06 app/settings.py:2  removed_from_HEAD_still_in_history
  [CRITICAL] aws-secret-access-key            ********(40 chars)       fp=d998a934ec0a x1 first=6b
c44ca07e06 app/settings.py:3  removed_from_HEAD_still_in_history
  [CRITICAL] stripe-secret-key                sk_t********(39 chars)   fp=d080eebd20d6 x1 first=20
8edc553b58 app/payments.py:1  active_in_HEAD
  [CRITICAL] private-key-block                ----********(35 chars)   fp=8529a64e32b5 x1 first=6d
c4a70a5d63 keys/deploy_key:1  removed_from_HEAD_still_in_history
  [HIGH    ] slack-token                      xoxb********(51 chars)   fp=0d6b029f76f3 x1 first=41
57ed52bdc9 app/notify.py:2  removed_from_HEAD_still_in_history
  [HIGH    ] slack-webhook                    http********(77 chars)   fp=7edd89cfbcef x1 first=41
57ed52bdc9 app/notify.py:1  active_in_HEAD
  [HIGH    ] google-api-key                   AIza********(39 chars)   fp=c8b47645db14 x1 first=20
8edc553b58 web/maps.js:1  active_in_HEAD
  [MEDIUM  ] generic-high-entropy-assignment  ********(27 chars)       fp=f4a10f8ea933 x1 first=20
8edc553b58 app/db.py:1  active_in_HEAD

$ python cli.py scan demo_repo --mode blobs
secretscan: blobs scan of demo_repo: 8 commits, 66 lines, 13 raw occurrences -> 11 unique secrets
(3 suppressed) in 12.3 ms

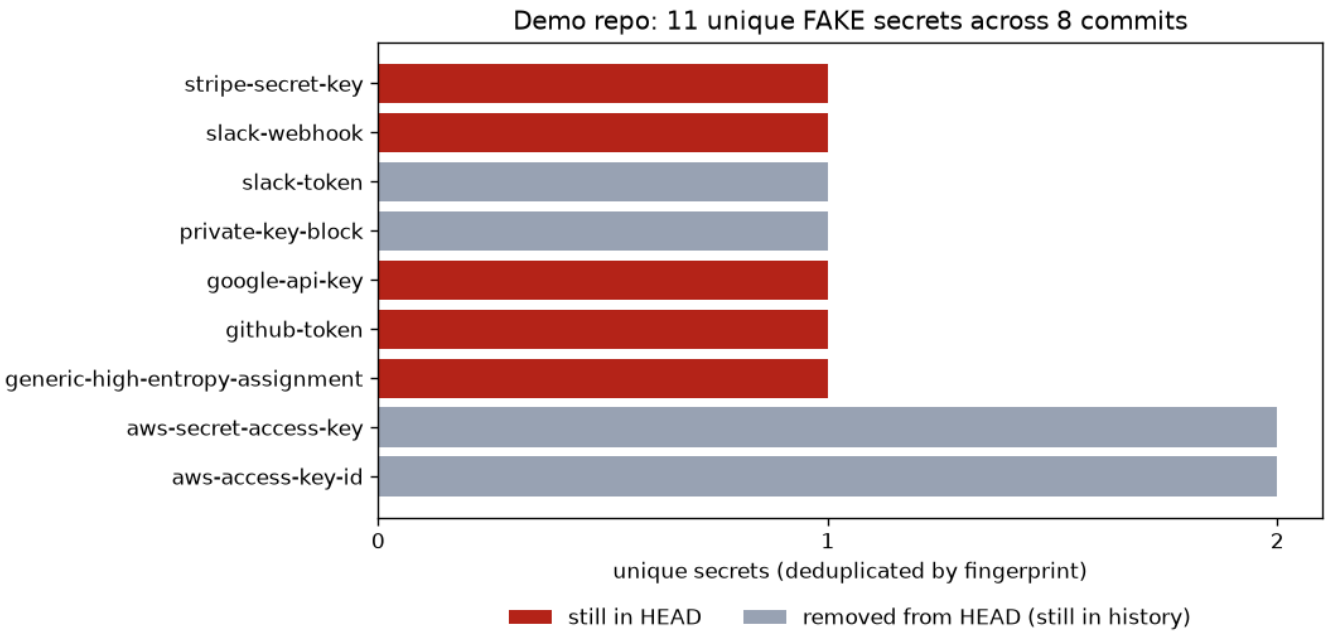
$ python cli.py install-hook hook_repo
installed pre-commit hook at hook_repo/.git/hooks/pre-commit

$ git commit -m 'clean commit'   -> exit 0
[master (root-commit) f3c4f81] clean commit
 1 file changed, 1 insertion(+)
 create mode 100644 app.py

$ git commit -m 'add stripe key'   -> exit 1
secretscan: commit BLOCKED - likely secrets in staged changes:
  config.py:1  stripe-secret-key  sk_t********(39 chars)  fp=d080eebd20d6
Remove them (use env vars / a secret manager), or allowlist a false positive with an inline
'secretscan:allow' comment or its fingerprint in .secretscan.yml. Bypass: git commit --no-verify

self-check: raw planted secrets found in report.json/report.html/report_blobs.json/console output:
 0

```


Demo repo: 11 unique FAKE secrets across 8 commits
stripe-secret-key
slack-webhook
slack-token
private-key-block
google-api-key
github-token
generic-high-entropy-assignment
aws-secret-access-key
aws-access-key-id
0
1
2
unique secrets (deduplicated by fingerprint)
still in HEAD
removed from HEAD (still in history)

09-git-secret-scanner/requirements.txt

```
GitPython>=3.1.40
PyYAML>=6.0
Jinja2>=3.1
streamlit>=1.30
plotly>=5.18
pandas>=2.0
matplotlib>=3.7
pytest>=7.4
pytest-timeout>=2.2

```

```
#!/usr/bin/env python3
"""secretscan CLI: generate-sample | scan | precommit | install-hook | uninstall-hook.

Exit codes: 0 = clean, 1 = secrets found (scan with --fail-on-findings, precommit), 2 = usage/error.
"""
from __future__ import annotations

import argparse
import sys
from pathlib import Path

from src.config import Config
from src.hook import install_hook, uninstall_hook
from src.report import build_report, format_console, write_html, write_json
from src.sample_repo import create_sample_repo
from src.scanner import scan_history, scan_staged


def cmd_generate(a: argparse.Namespace) -> int:
    repo = create_sample_repo(a.dest, overwrite=a.force)
    commits = list(repo.iter_commits("--all"))
    print(f"created demo repo at {a.dest} with {len(commits)} commits (all planted secrets are FAKE examples)")
    for c in reversed(commits):
        print(f"  {c.hexsha[:12]}  {c.summary}")
    return 0


def cmd_scan(a: argparse.Namespace) -> int:
    cfg = Config.load(a.config, repo_root=a.repo) if (a.config or a.repo) else Config()
    if a.no_repo_config and not a.config:
        cfg = Config()
    res = scan_history(a.repo, cfg, mode=a.mode, rev=a.rev, max_commits=a.max_commits)
    print(format_console(res))
    report = build_report(res)
    if a.json:
        print(f"JSON report -> {write_json(report, a.json)}")
    if a.html:
        print(f"HTML report -> {write_html(report, a.html)}")
    return 1 if (a.fail_on_findings and res.findings) else 0


def cmd_precommit(a: argparse.Namespace) -> int:
    res = scan_staged(a.repo, Config.load(a.config, repo_root=a.repo))
    if not res.findings:
        print(f"secretscan: staged changes clean ({res.lines_scanned} added lines, {len(res.suppressed)} suppressed)")
        return 0
    print("secretscan: commit BLOCKED - likely secrets in staged changes:", file=sys.stderr)
    for f in res.findings:
        print(f"  {f.path}:{f.line}  {f.rule_id}  {f.redacted}  fp={f.fingerprint[:12]}", file=sys.stderr)
    print("Remove them (use env vars / a secret manager), or allowlist a false positive with an inline\n"
          "'secretscan:allow' comment or its fingerprint in .secretscan.yml. Bypass: git commit --no-verify",
          file=sys.stderr)
    return 1


def cmd_install(a: argparse.Namespace) -> int:
    p = install_hook(a.repo, force=a.force)
    print(f"installed pre-commit hook at {p}")
    return 0
```

```

def cmd_uninstall(a: argparse.Namespace) -> int:
    print("removed secretscan hook" if uninstall_hook(a.repo) else "no secretscan hook found")
    return 0


def build_parser() -> argparse.ArgumentParser:
    ap = argparse.ArgumentParser(prog="secretscan", description="Git-history secret leak detection engine")
    sub = ap.add_subparsers(dest="cmd", required=True)

    g = sub.add_parser("generate-sample", help="create a throwaway demo repo with planted FAKE secrets")
    g.add_argument("dest", nargs="?", default="sample_repo")
    g.add_argument("--force", action="store_true", help="delete and recreate dest if it exists")
    g.set_defaults(func=cmd_generate)

    s = sub.add_parser("scan", help="scan every commit of a repository's history")
    s.add_argument("repo", nargs="?", default=".")
    s.add_argument("--mode", choices=["patch", "blobs"], default="patch")
    s.add_argument("--rev", help="limit to a revision/range instead of --all refs")
    s.add_argument("--max-commits", type=int)
    s.add_argument("--config", help="YAML config (default: <repo>/.secretscan.yml if present)")
    s.add_argument("--no-repo-config", action="store_true", help="ignore the repo's .secretscan.yml")
    s.add_argument("--json", help="write JSON report here")
    s.add_argument("--html", help="write HTML report here")
    s.add_argument("--fail-on-findings", action="store_true", help="exit 1 if anything is reported (CI)")
    s.set_defaults(func=cmd_scan)

    p = sub.add_parser("precommit", help="scan staged changes; exit 1 if a secret is found (used by the hook)")
    p.add_argument("--repo", default=".")
    p.add_argument("--config")
    p.set_defaults(func=cmd_precommit)

    i = sub.add_parser("install-hook", help="install the git pre-commit hook into a repo")
    i.add_argument("repo", nargs="?", default=".")
    i.add_argument("--force", action="store_true", help="back up and replace an existing foreign hook")
    i.set_defaults(func=cmd_install)

    u = sub.add_parser("uninstall-hook", help="remove the secretscan hook (restores a backup if any)")
    u.add_argument("repo", nargs="?", default=".")
    u.set_defaults(func=cmd_uninstall)
    return ap


def main(argv: list[str] | None = None) -> int:
    a = build_parser().parse_args(argv)
    try:
        return a.func(a)
    except (FileExistsError, ValueError) as e:
        print(f"error: {e}", file=sys.stderr)
        return 2
    except Exception as e:  # git errors etc.
        print(f"error: {type(e).__name__}: {e}", file=sys.stderr)
        return 2


if __name__ == "__main__":
    sys.exit(main())
```

```
118
```

```
"""Streamlit dashboard for secretscan reports.

Reads outputs/report.json (bundled, generated from the FAKE demo repo) or a report you
upload. Works fully offline; it never shows raw secrets because reports never contain them.
Run:  streamlit run app.py
"""
from __future__ import annotations

import json
from pathlib import Path

import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parent
DEFAULT_REPORT = ROOT / "outputs" / "report.json"
PUBLIC_SCANS = ROOT / "outputs" / "public_repo_scans.json"

st.set_page_config(page_title="secretscan dashboard", layout="wide")
st.title("secretscan: git history secret leak report")
st.caption("Values are redacted at scan time; the dashboard only ever sees fingerprints and redacted prefixes.")

uploaded = st.sidebar.file_uploader("Load another secretscan JSON report", type=["json"])
if uploaded is not None:
    report = json.load(uploaded)
elif DEFAULT_REPORT.exists():
    report = json.loads(DEFAULT_REPORT.read_text())
else:
    st.warning("No report found. Run `python scripts/run_demo.py` or `python cli.py scan <repo> --json outputs/report.json`.")
    st.stop()

stats = report["stats"]
secrets = pd.DataFrame(report["secrets"])
sev_filter = st.sidebar.multiselect("Severity", ["critical", "high", "medium", "low"],
                                    default=["critical", "high", "medium", "low"])
status_filter = st.sidebar.radio("Status", ["all", "active_in_HEAD", "removed_from_HEAD_still_in_history"])

tab_over, tab_find, tab_time, tab_sup, tab_pub = st.tabs(
    ["Overview", "Findings", "Timeline", "Suppressed", "Public repo scans"])

with tab_over:
    c = st.columns(6)
    c[0].metric("Commits scanned", stats["commits_scanned"])
    c[1].metric("Raw occurrences", stats["raw_occurrences"])
    c[2].metric("Unique secrets", stats["unique_secrets"])
    c[3].metric("Still in HEAD", stats["active_in_head"])
    c[4].metric("Removed, in history", stats["removed_but_in_history"])
    c[5].metric("Suppressed", stats["suppressed_occurrences"])
    st.write(f"Target **{report['target']}** · mode **{report['mode']}** · {stats['lines_scanned']} lines · "
             f"{stats['duration_ms']} ms")
    if not secrets.empty:
        by_rule = secrets.groupby(["rule_id", "status"]).size().reset_index(name="count")
        st.plotly_chart(px.bar(by_rule, x="count", y="rule_id", color="status", orientation="h",
                               title="Unique secrets by rule"), width="stretch")
        sev = secrets["severity"].value_counts().reset_index()
        sev.columns = ["severity", "count"]
        st.plotly_chart(px.pie(sev, names="severity", values="count", title="By severity", hole=0.4),
                        width="stretch")

```

```
with tab_find:
    if secrets.empty:
        st.success("No secrets found.")
    else:
        df = secrets[secrets["severity"].isin(sev_filter)]
        if status_filter != "all":
            df = df[df["status"] == status_filter]
        view = df.assign(first_commit=df["first_seen"].map(lambda d: d["commit"]),
                         first_location=df["first_seen"].map(lambda d: f"{d['path']}:{d['line']}")
,
                         files=df["files"].map(", ".join))
        st.dataframe(view[["severity", "rule_id", "redacted", "fingerprint", "status", "occurrence
_count",
                           "first_commit", "first_location", "files", "entropy"]],
                     width="stretch", hide_index=True)
        pick = st.selectbox("Occurrences for fingerprint", view["fingerprint"].tolist())
        if pick:
            occ = view.loc[view["fingerprint"] == pick, "occurrences"].iloc[0]
            st.table(pd.DataFrame(occ))

with tab_time:
    rows = [dict(o, rule_id=s["rule_id"], fingerprint=s["fingerprint"][:12])
            for s in report["secrets"] for o in s["occurrences"]]
    if rows:
        tl = pd.DataFrame(rows)
        tl["date"] = pd.to_datetime(tl["date"])
        st.plotly_chart(px.scatter(tl, x="date", y="rule_id", color="fingerprint", hover_data=["co
mmit", "path", "line"],
                                   title="When each secret entered history"), width="stretch")
        per_commit = tl.groupby(["commit", "date"]).size().reset_index(name="new_occurrences").sor
t_values("date")
        st.dataframe(per_commit, hide_index=True, width="stretch")
    else:
        st.info("Nothing to plot.")

with tab_sup:
    sup = pd.DataFrame(report.get("suppressed", []))
    if sup.empty:
        st.info("Nothing was suppressed.")
    else:
        st.write("Matches skipped by the allowlist (inline comment, path glob, fingerprint or valu
e regex):")
        st.dataframe(sup, hide_index=True, width="stretch")

with tab_pub:
    if PUBLIC_SCANS.exists():
        pub = pd.DataFrame(json.loads(PUBLIC_SCANS.read_text()))
        st.write("Default-config history scans of real public repositories (measured run; see READ
ME).")
        st.dataframe(pub[["repo", "head_date", "commits_scanned", "added_lines_scanned", "raw_occu
rrences",
                          "unique_findings", "duration_s"]], hide_index=True, width="stretch")
        for _, r in pub.iterrows():
            if r["findings"]:
                with st.expander(f"{r['repo']}: {len(r['findings'])} unique findings"):
                    st.table(pd.DataFrame(r["findings"]))
    else:
        st.info("Run `python scripts/scan_public_repos.py` to add real-repo measurements.")

```

09-git-secret-scanner/src/__init__.py

```python
"""secretscan - a git-history secret leak detection engine (educational / defensive)."""
__version__ = "1.0.0"

```

```
"""Scanner configuration and allowlist (loaded from YAML, e.g. ``.secretscan.yml``).

Example::

    entropy_threshold: 3.5        # bits/char gate for the generic rule
    min_generic_length: 16
    disabled_rules: []            # rule ids to switch off
    allowlist:
      paths: ["tests/fixtures/*"] # fnmatch globs on repo-relative paths
      fingerprints: ["3f1c..."]   # fingerprints copied from a previous report
      values_regex: []            # regexes matched against the secret value (use sparingly)

Inline suppression: a line containing ``secretscan:allow`` or
``pragma: allowlist secret`` is never reported.
"""
from __future__ import annotations

import fnmatch
import re
from dataclasses import dataclass, field
from pathlib import Path
from typing import Optional

import yaml

DEFAULT_CONFIG_NAME = ".secretscan.yml"
INLINE_MARKERS = ("secretscan:allow", "pragma: allowlist secret")


@dataclass
class Config:
    entropy_threshold: float = 3.5
    min_generic_length: int = 16
    disabled_rules: list[str] = field(default_factory=list)
    allow_paths: list[str] = field(default_factory=list)
    allow_fingerprints: list[str] = field(default_factory=list)
    allow_value_regex: list[str] = field(default_factory=list)
    max_line_length: int = 5000  # longer lines (minified bundles) are cut before regexes run

    def __post_init__(self) -> None:
        self._value_res = [re.compile(p) for p in self.allow_value_regex]

    # --- allowlist decisions ------------------------------------------------
    def path_allowed(self, path: str) -> bool:
        return any(fnmatch.fnmatch(path, g) for g in self.allow_paths)

    def fingerprint_allowed(self, fp: str) -> bool:
        return any(fp.startswith(a) for a in self.allow_fingerprints if len(a) >= 8)

    def value_allowed(self, value: str) -> bool:
        return any(r.search(value) for r in self._value_res)

    @staticmethod
    def inline_suppressed(line: str) -> bool:
        low = line.lower()
        return any(m in low for m in INLINE_MARKERS)

    # --- loading ------------------------------------------------------------
    @classmethod
    def from_dict(cls, d: dict) -> "Config":
        allow = d.get("allowlist") or {}
        return cls(
            entropy_threshold=float(d.get("entropy_threshold", 3.5)),
            min_generic_length=int(d.get("min_generic_length", 16)),
            disabled_rules=list(d.get("disabled_rules") or []),
```

09-git-secret-scanner/src/config.py (continued)

```
            allow_paths=list(allow.get("paths") or []),
            allow_fingerprints=[str(x) for x in (allow.get("fingerprints") or [])],
            allow_value_regex=list(allow.get("values_regex") or []),
        )

    @classmethod
    def load(cls, path: Optional[str | Path] = None, repo_root: Optional[str | Path] = None) -> "C
onfig":
        """Load an explicit config file, else ``<repo_root>/.secretscan.yml``, else defaults."""
        candidate: Optional[Path] = Path(path) if path else None
        if candidate is None and repo_root is not None:
            p = Path(repo_root) / DEFAULT_CONFIG_NAME
            candidate = p if p.is_file() else None
        if candidate is None:
            return cls()
        data = yaml.safe_load(candidate.read_text(encoding="utf-8")) or {}
        if not isinstance(data, dict):
            raise ValueError(f"config {candidate} must be a YAML mapping")
        return cls.from_dict(data)

```

09-git-secret-scanner/src/entropy.py

```
"""Shannon entropy helpers used to judge whether a string looks random (key-like)."""
from __future__ import annotations

import math
from collections import Counter


def shannon_entropy(value: str) -> float:
    """Return the Shannon entropy of ``value`` in bits per character.

    H = -sum(p_i * log2(p_i)) over the distinct characters of the string.
    English words sit around 2.5-3.5 bits/char; random base64 tokens of
    length 20-40 sit around 4.0-5.2 bits/char.
    """
    if not value:
        return 0.0
    counts = Counter(value)
    n = len(value)
    return -sum((c / n) * math.log2(c / n) for c in counts.values())


def char_classes(value: str) -> int:
    """How many of the classes lower / upper / digit / symbol appear in ``value``."""
    return sum([
        any(ch.islower() for ch in value),
        any(ch.isupper() for ch in value),
        any(ch.isdigit() for ch in value),
        any(not ch.isalnum() for ch in value),
    ])

```

```
"""Install / uninstall a git pre-commit hook that runs ``cli.py precommit``."""
from __future__ import annotations

import os
import shlex
import stat
import sys
from pathlib import Path
from typing import Optional

import git

MARKER = "# installed-by: secretscan"
CLI_PATH = Path(__file__).resolve().parent.parent / "cli.py"


def hook_path(repo_path: str | Path) -> Path:
    repo = git.Repo(repo_path)
    hooks_dir = Path(repo.git.rev_parse("--git-path", "hooks"))
    if not hooks_dir.is_absolute():
        hooks_dir = Path(repo.working_tree_dir) / hooks_dir
    return hooks_dir / "pre-commit"


def hook_script(python: str, cli: str) -> str:
    return (
        "#!/bin/sh\n"
        f"{MARKER}\n"
        "# Blocks the commit (exit 1) when staged changes contain a likely secret.\n"
        "# Bypass deliberately with: git commit --no-verify\n"
        f"exec {shlex.quote(python)} {shlex.quote(cli)} precommit --repo \"$(git rev-parse --show-toplevel)\"\n"
    )


def install_hook(repo_path: str | Path, force: bool = False, python: Optional[str] = None,
                 cli: Optional[str] = None) -> Path:
    """Write the hook. An existing foreign hook is kept unless ``force`` (then backed up)."""
    p = hook_path(repo_path)
    p.parent.mkdir(parents=True, exist_ok=True)
    if p.exists() and MARKER not in p.read_text(errors="replace"):
        if not force:
            raise FileExistsError(f"{p} exists and was not installed by secretscan (use --force to back it up and replace)")
        p.replace(p.with_name("pre-commit.secretscan.bak"))
    p.write_text(hook_script(python or sys.executable, cli or str(CLI_PATH)))
    p.chmod(p.stat().st_mode | stat.S_IXUSR | stat.S_IXGRP | stat.S_IXOTH)
    return p


def uninstall_hook(repo_path: str | Path) -> bool:
    p = hook_path(repo_path)
    if p.exists() and MARKER in p.read_text(errors="replace"):
        p.unlink()
        bak = p.with_name("pre-commit.secretscan.bak")
        if bak.exists():
            bak.replace(p)
        return True
    return False

```

09-git-secret-scanner/src/redact.py

```
"""Fingerprinting and redaction: reports never carry a raw secret."""
from __future__ import annotations

import hashlib

FINGERPRINT_PREFIX = "secretscan:v1:"


def fingerprint(secret: str) -> str:
    """Stable identifier for a secret value (SHA-256, first 24 hex chars).

    Used for dedupe across commits/files and for allowlisting. It is a one-way
    hash, but a short or low-entropy secret could be brute-forced from it, so
    reports should still be treated as sensitive.
    """
    return hashlib.sha256((FINGERPRINT_PREFIX + secret).encode("utf-8")).hexdigest()[:24]


# Documented, non-secret type prefixes that are safe (and useful) to show in a report.
PUBLIC_PREFIXES = ("AKIA", "ASIA", "AGPA", "AIDA", "AROA", "ANPA", "ANVA", "ghp_", "gho_", "ghu_",
 "ghs_",
                   "ghr_", "github_pat_", "xoxb", "xoxa", "xoxp", "xoxr", "xoxs", "sk_live_", "sk_
test_",
                   "rk_live_", "rk_test_", "AIza", "https://hooks.slack.com/", "-----BEGIN")


def redact(secret: str) -> str:
    """Return a display form that never reveals secret characters.

    Only a documented public type prefix (e.g. ``AKIA``, ``ghp_``) is kept, capped
    at 4 characters; generic secrets (passwords etc.) show no characters at all.
    """
    keep = 4 if secret.startswith(PUBLIC_PREFIXES) else 0
    return f"{secret[:keep]}{'*' * 8}({len(secret)} chars)"

```

```
"""JSON and self-contained HTML reports. Only redacted values and fingerprints are written."""
from __future__ import annotations

import json
from collections import Counter
from datetime import datetime, timezone
from pathlib import Path

from jinja2 import Environment, select_autoescape

from . import __version__
from .scanner import ScanResult


def build_report(res: ScanResult) -> dict:
    unique = res.unique()
    by_rule = Counter(u["rule_id"] for u in unique)
    by_sev = Counter(u["severity"] for u in unique)
    by_status = Counter(u["status"] for u in unique)
    return {
        "tool": "secretscan",
        "version": __version__,
        "generated_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "target": res.target,
        "mode": res.mode,
        "stats": {
            "commits_scanned": res.commits_scanned,
            "blobs_scanned": res.blobs_scanned,
            "lines_scanned": res.lines_scanned,
            "raw_occurrences": len(res.findings),
            "unique_secrets": len(unique),
            "suppressed_occurrences": len(res.suppressed),
            "active_in_head": by_status.get("active_in_HEAD", 0),
            "removed_but_in_history": by_status.get("removed_from_HEAD_still_in_history", 0),
            "by_rule": dict(sorted(by_rule.items())),
            "by_severity": dict(by_sev),
            "duration_ms": res.duration_ms,
        },
        "secrets": unique,
        "suppressed": [
            {"fingerprint": f.fingerprint, "rule_id": f.rule_id, "redacted": f.redacted,
             "reason": f.suppressed_by, "path": f.path, "line": f.line, "commit": f.commit}
            for f in res.suppressed
        ],
        "notes": [
            "Raw secret values are never stored; 'redacted' shows at most a 4-char prefix.",
            "Secrets removed from HEAD are still in git history and must be rotated/revoked.",
        ],
    }


def write_json(report: dict, path: str | Path) -> Path:
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps(report, indent=2), encoding="utf-8")
    return p


_HTML = """<!doctype html>
<html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>secretscan report - {{ r.target }}</title>
<style>
 body{font-family:system-ui,-apple-system,Segoe UI,Roboto,sans-serif;margin:24px;color:#1f2933;bac
kground:#f7f9fb}
```

```
 h1{font-size:1.5rem;margin:0 0 4px} .sub{color:#61707f;margin-bottom:18px}
 .cards{display:flex;flex-wrap:wrap;gap:10px;margin-bottom:18px}
 .card{background:#fff;border:1px solid #dde3ea;border-radius:8px;padding:10px 14px;min-width:120px}
 .card b{display:block;font-size:1.4rem}
 table{border-collapse:collapse;width:100%;background:#fff;font-size:.88rem;margin-bottom:20px}
 th,td{border:1px solid #dde3ea;padding:6px 8px;text-align:left;vertical-align:top}
 th{background:#eef2f6} code{font-family:ui-monospace,Menlo,Consolas,monospace;font-size:.85rem}
 .sev-critical{color:#fff;background:#b42318;padding:1px 6px;border-radius:4px}
 .sev-high{color:#fff;background:#dc6803;padding:1px 6px;border-radius:4px}
 .sev-medium{color:#1f2933;background:#fdb022;padding:1px 6px;border-radius:4px}
 .active{color:#b42318;font-weight:600} .removed{color:#61707f}
</style></head><body>
<h1>Secret scan report: {{ r.target }}</h1>
<div class="sub">secretscan {{ r.version }} &middot; mode {{ r.mode }} &middot; generated {{ r.generated_at }} &middot; values are redacted</div>
<div class="cards">
 <div class="card"><b>{{ r.stats.commits_scanned }}</b>commits</div>
 <div class="card"><b>{{ r.stats.raw_occurrences }}</b>raw occurrences</div>
 <div class="card"><b>{{ r.stats.unique_secrets }}</b>unique secrets</div>
 <div class="card"><b>{{ r.stats.active_in_head }}</b>still in HEAD</div>
 <div class="card"><b>{{ r.stats.removed_but_in_history }}</b>removed, in history</div>
 <div class="card"><b>{{ r.stats.suppressed_occurrences }}</b>suppressed</div>
</div>
<h2>Unique secrets</h2>
<table><tr><th>Severity</th><th>Rule</th><th>Redacted</th><th>Fingerprint</th><th>Status</th><th>First seen</th><th>Occurrences</th></tr>
{% for s in r.secrets %}
<tr><td><span class="sev-{{ s.severity }}">{{ s.severity }}</span></td><td>{{ s.rule_id }}</td>
<td><code>{{ s.redacted }}</code></td><td><code>{{ s.fingerprint }}</code></td>
<td class="{{ 'active' if s.status == 'active_in_HEAD' else 'removed' }}">{{ s.status }}</td>
<td><code>{{ s.first_seen.commit }}</code> {{ s.first_seen.path }}:{{ s.first_seen.line }}</td>
<td>{{ s.occurrence_count }}{% for o in s.occurrences %}<br><code>{{ o.commit }}</code> {{ o.path }}:{{ o.line }}{% endfor %}</td></tr>
{% else %}<tr><td colspan="7">No secrets found.</td></tr>{% endfor %}
</table>
<h2>Suppressed (allowlisted)</h2>
<table><tr><th>Rule</th><th>Redacted</th><th>Reason</th><th>Location</th></tr>
{% for s in r.suppressed %}<tr><td>{{ s.rule_id }}</td><td><code>{{ s.redacted }}</code></td><td>{{ s.reason }}</td><td><code>{{ s.commit }}</code> {{ s.path }}:{{ s.line }}</td></tr>
{% else %}<tr><td colspan="4">None.</td></tr>{% endfor %}
</table>
<p class="sub">Secrets removed from HEAD remain in git history: revoke/rotate them at the provider, then rewrite history if needed.</p>
</body></html>"""


def write_html(report: dict, path: str | Path) -> Path:
    env = Environment(autoescape=select_autoescape(default=True, default_for_string=True))
    html = env.from_string(_HTML).render(r=report)
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(html, encoding="utf-8")
    return p


def format_console(res: ScanResult) -> str:
    """Human-readable, redacted summary for the terminal."""
    rows = res.unique()
    out = [f"secretscan: {res.mode} scan of {res.target}: {res.commits_scanned} commits, "
           f"{res.lines_scanned} lines, {len(res.findings)} raw occurrences -> {len(rows)} unique secrets "
           f"({len(res.suppressed)} suppressed) in {res.duration_ms} ms"]
    for r in rows:
        fs = r["first_seen"]
```

09-git-secret-scanner/src/report.py (continued)

```
        out.append(f"  [{r['severity'].upper():8}] {r['rule_id']:32} {r['redacted']:24} fp={r['fingerprint'][:12]} "
                   f"x{r['occurrence_count']} first={fs['commit']} {fs['path']}:{fs['line']}  {r['status']}")
    return "\n".join(out)

```

```
"""Detection rules: provider-specific regexes plus a generic entropy-gated rule.

Each rule names a regex whose *first capturing group* (or the whole match when
there is no group) is the secret value. Provider rules match on the key's
documented shape (prefix + length + alphabet). They do NOT validate keys
against any provider API - this tool never sends a candidate secret anywhere.
"""
from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Optional

from .entropy import char_classes, shannon_entropy


@dataclass(frozen=True)
class Rule:
    rule_id: str
    description: str
    severity: str  # critical | high | medium | low
    pattern: re.Pattern
    min_entropy: float = 0.0          # extra gate for noisy shapes
    keyword_hint: Optional[str] = None  # cheap pre-filter: skip the regex if absent
    tags: tuple = field(default_factory=tuple)

    def secret_from(self, m: re.Match) -> str:
        return m.group(1) if m.groups() else m.group(0)


def _r(p: str, flags: int = 0) -> re.Pattern:
    return re.compile(p, flags)


# Provider rules. Shapes follow the providers' published key formats.
RULES: list[Rule] = [
    Rule("aws-access-key-id", "AWS access key ID", "critical",
         _r(r"\b((?:AKIA|ASIA|AGPA|AIDA|AROA|ANPA|ANVA)[0-9A-Z]{16})\b"), keyword_hint="A"),
    Rule("aws-secret-access-key", "AWS secret access key (assignment)", "critical",
         _r(r"(?i)aws.{0,20}?(?:secret|sk).{0,20}?['\"=:\s]+([A-Za-z0-9/+=]{40})(?![A-Za-z0-9/+=])
"),
         min_entropy=3.5, keyword_hint="aws"),
    Rule("github-token", "GitHub personal access / OAuth / app token", "critical",
         _r(r"\b((?:ghp|gho|ghu|ghs|ghr)_[A-Za-z0-9]{36})\b"), keyword_hint="gh"),
    Rule("github-fine-grained-pat", "GitHub fine-grained personal access token", "critical",
         _r(r"\b(github_pat_[A-Za-z0-9_]{82})\b"), keyword_hint="github_pat_"),
    Rule("slack-token", "Slack bot/user/app token", "high",
         _r(r"\b(xox[baprs]-[A-Za-z0-9-]{10,72})\b"), keyword_hint="xox"),
    Rule("slack-webhook", "Slack incoming webhook URL", "high",
         _r(r"(https://hooks\.slack\.com/services/T[A-Za-z0-9_]{8,12}/B[A-Za-z0-9_]{8,12}/[A-Za-z0
-9_]{20,32})"),
         keyword_hint="hooks.slack.com"),
    Rule("stripe-secret-key", "Stripe secret / restricted key", "critical",
         _r(r"\b((?:sk|rk)_(?:live|test)_[A-Za-z0-9]{16,99})\b"), keyword_hint="k_"),
    Rule("google-api-key", "Google API key", "high",
         _r(r"\b(AIza[0-9A-Za-z_\-]{35})(?![0-9A-Za-z_\-])"), keyword_hint="AIza"),
    Rule("private-key-block", "PEM private key block header", "critical",
         _r(r"(-----BEGIN (?:RSA |EC |DSA |OPENSSH |PGP |ENCRYPTED )?PRIVATE KEY(?: BLOCK)?-----)"
),
         keyword_hint="PRIVATE KEY"),
]

# Generic rule: a credential-ish variable name assigned a quoted literal.
GENERIC_ASSIGNMENT = _r(
    r"""(?ix)
```

```
    (?P<name>[a-z0-9_.\-]*(?:password|passwd|pwd|secret|token|api[_-]?key|apikey|
             access[_-]?key|auth[_-]?key|client[_-]?secret|private[_-]?key|credential)s?[a-z0-9_.\
-]*)
    \s*(?::=|=>|=|:)\s*
    (?P<q>['"`])(?P<value>[^'"`\s]{12,200})(?P=q)
    """
)

GENERIC_RULE_ID = "generic-high-entropy-assignment"

# Values that are obviously placeholders / templates, never reported by the generic rule.
_PLACEHOLDER_RE = _r(
    r"""(?ix)^(?:
        \$\{.*\}|\$[A-Z_]+|\{\{.*\}\}|<[^>]+>|%\(.*\)s|%s|
        (?:x|\*|0|\.){6,}|
        (?:changeme|change_me|password|secret|your[_-]?.*|replace[_-]?me|dummy|null|none|todo|reda
cted)
    )$"""
)


def is_placeholder(value: str) -> bool:
    return bool(_PLACEHOLDER_RE.match(value)) or value.lower().startswith(("os.environ", "env("))


def generic_candidates(line: str, min_entropy: float, min_length: int = 16):
    """Yield (secret, entropy, start, end) for generic high-entropy assignments."""
    for m in GENERIC_ASSIGNMENT.finditer(line):
        value = m.group("value")
        if len(value) < min_length or is_placeholder(value):
            continue
        h = shannon_entropy(value)
        if h >= min_entropy and char_classes(value) >= 3:
            yield value, h, m.start("value"), m.end("value")

```

```
"""Generate a throwaway demo git repository with planted FAKE secrets.

Every planted value is visibly fake: it is either a value AWS/Slack publish in
their own documentation as an example, or contains FAKE / EXAMPLE / TEST and
runs of zeros. None of them can authenticate anywhere. The values are
assembled from fragments at runtime so that this source file itself never
contains a contiguous key-shaped string (so pushing this project to GitHub
does not trip GitHub's push protection, and the scanner does not flag its own
source).

The history (fixed dates => deterministic commit SHAs):
  c1 initial skeleton (clean)
  c2 hardcode AWS key id + secret in settings.py and duplicate the id in deploy/env.sh
  c3 add Slack webhook, Slack bot token, GitHub token (CI script)
  c4 "rotate" the AWS key - the new key is hardcoded again
  c5 move AWS creds to env vars, delete deploy/env.sh, drop the Slack bot token
  c6 add payments (Stripe test key), maps (Google key), DB password (generic
     high-entropy), placeholders, an inline-suppressed token, a fixture under an
     allowlisted path, a docs key allowlisted by fingerprint, and .secretscan.yml
  c7 commit a (fake) private key file
  c8 delete the private key file
"""
from __future__ import annotations

import shutil
from datetime import datetime, timedelta, timezone
from pathlib import Path

import git

from .redact import fingerprint


def _j(*parts: str) -> str:
    return "".join(parts)


# label -> fake value.  (Lengths are asserted in tests.)
FAKE = {
    "aws_key_1": _j("AKIA", "IOSFODNN7", "EXAMPLE"),                        # AWS docs example id
    "aws_secret_1": _j("wJalrXUtnFEMI/K7MDENG/", "bPxRfiCY", "EXAMPLEKEY"),  # AWS docs example se
cret
    "aws_key_2": _j("AKIA", "FAKEEXAMPLE", "00002"),
    "aws_secret_2": _j("Fake/Example+Rotated", "Secret0Key/DoNotUse0002"[:20]),
    "slack_webhook": _j("https://hooks.", "slack.com/services/", "T00000000/B00000000/", "X" * 24)
,  # Slack docs example
    "slack_token": _j("xo", "xb-", "000000000000-000000000000-", "FAKEEXAMPLETOKEN0000"),
    "github_token": _j("gh", "p_", "FAKEEXAMPLETOKENDONOTUSE", "0" * 12),
    "stripe_key": _j("sk", "_test_", "FAKEEXAMPLE", "0" * 20),
    "google_key": _j("AI", "za", "FAKE_EXAMPLE_KEY_", "0" * 18),
    "db_password": _j("Xq7#fake", "R9$mLp2!", "Vt4wEXAMPLE"),
    "inline_token": _j("Tr4!nxFAKE", "8wQz#2LmPv", "EXAMPLE"),
    "fixture_token": _j("gh", "p_", "TESTFIXTUREFAKEEXAMPLE00", "0" * 12),
    "docs_google_key": _j("AI", "za", "PUBLIC_DOCS_DEMO_FAKE_", "0" * 13),
    "private_key_header": _j("-----BEGIN ", "OPENSSH PRIVATE", " KEY-----"),
}

# Ground truth for tests / measured metrics.
#   rule, expected outcome ("reported" | suppressed reason), status if reported
EXPECTED = {
    "aws_key_1": ("aws-access-key-id", "reported", "removed"),
    "aws_secret_1": ("aws-secret-access-key", "reported", "removed"),
    "aws_key_2": ("aws-access-key-id", "reported", "removed"),
    "aws_secret_2": ("aws-secret-access-key", "reported", "removed"),
    "slack_webhook": ("slack-webhook", "reported", "active"),
```

```
    "slack_token": ("slack-token", "reported", "removed"),
    "github_token": ("github-token", "reported", "active"),
    "stripe_key": ("stripe-secret-key", "reported", "active"),
    "google_key": ("google-api-key", "reported", "active"),
    "db_password": ("generic-high-entropy-assignment", "reported", "active"),
    "private_key_header": ("private-key-block", "reported", "removed"),
    "inline_token": ("generic-high-entropy-assignment", "inline", None),
    "fixture_token": ("github-token", "path", None),
    "docs_google_key": ("google-api-key", "fingerprint", None),
}

# Benign lines that must NOT be reported (placeholders, env lookups, low entropy, hashes of words).
BENIGN_LINES = [
    'password = "changeme"',
    'api_token = "${API_TOKEN}"',
    'secret_key = os.environ["SECRET_KEY"]',
    'db_password = "<your-db-password>"',
    'token = "xxxxxxxxxxxxxxxxxxxx"',
    'description = "this is a perfectly normal sentence"',
    'auth_header = "Bearer " + token',
    'password_hint = "favouritecolour"',
]

AUTHOR = git.Actor("Demo Developer", "demo.dev@example.com")
BASE_DATE = datetime(2026, 1, 5, 9, 0, 0, tzinfo=timezone.utc)


def _write(root: Path, rel: str, text: str) -> str:
    p = root / rel
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(text, encoding="utf-8")
    return rel


def create_sample_repo(dest: str | Path, overwrite: bool = False) -> git.Repo:
    """Create the demo repository at ``dest`` and return it."""
    dest = Path(dest)
    if dest.exists():
        if not overwrite:
            raise FileExistsError(f"{dest} exists (pass overwrite=True / --force)")
        shutil.rmtree(dest)
    dest.mkdir(parents=True)
    repo = git.Repo.init(dest, initial_branch="main") if _supports_initial_branch() else git.Repo.init(dest)
    n = 0

    def commit(msg: str, add: list[str] = (), remove: list[str] = ()) -> None:
        nonlocal n
        n += 1
        if add:
            repo.index.add(list(add))
        if remove:
            repo.index.remove(list(remove), working_tree=True)
        when = (BASE_DATE + timedelta(days=n)).strftime("%Y-%m-%dT%H:%M:%S+0000")
        repo.index.commit(msg, author=AUTHOR, committer=AUTHOR, author_date=when, commit_date=when)

    F = FAKE
    commit("Initial skeleton", [
        _write(dest, "README.md", "# demo-shop\nA tiny demo app used to exercise secretscan.\nALL credentials in this history are FAKE examples.\n"),
        _write(dest, "app/main.py", "def main():\n    print('hello shop')\n\n\nif __name__ == '__main__':\n    main()\n"),
    ])
```

```
    commit("Add AWS settings for S3 uploads", [
        _write(dest, "app/settings.py",
               "# S3 upload settings (FAKE demo values)\n"
               f'AWS_ACCESS_KEY_ID = "{F["aws_key_1"]}"\n'
               f'AWS_SECRET_ACCESS_KEY = "{F["aws_secret_1"]}"\n'
               'BUCKET = "demo-shop-uploads"\n'),
        _write(dest, "deploy/env.sh", f"#!/bin/sh\nexport AWS_ACCESS_KEY_ID={F['aws_key_1']}\nexport REGION=eu-west-1\n"),
    ])
    commit("Add Slack notifications and CI script", [
        _write(dest, "app/notify.py",
               f'SLACK_WEBHOOK = "{F["slack_webhook"]}"\n'
               f'SLACK_BOT_TOKEN = "{F["slack_token"]}"\n\n'
               "def notify(msg):\n    return {'text': msg}\n"),
        _write(dest, "scripts/ci.sh", f"#!/bin/sh\n# clone private deps\nGH_TOKEN={F['github_token']}\necho building\n"),
    ])
    commit("Rotate AWS key", [
        _write(dest, "app/settings.py",
               "# S3 upload settings (FAKE demo values) - rotated\n"
               f'AWS_ACCESS_KEY_ID = "{F["aws_key_2"]}"\n'
               f'AWS_SECRET_ACCESS_KEY = "{F["aws_secret_2"]}"\n'
               'BUCKET = "demo-shop-uploads"\n'),
    ])
    commit("Move AWS credentials to environment variables", [
        _write(dest, "app/settings.py",
               "import os\n"
               'AWS_ACCESS_KEY_ID = os.environ["AWS_ACCESS_KEY_ID"]\n'
               'AWS_SECRET_ACCESS_KEY = os.environ["AWS_SECRET_ACCESS_KEY"]\n'
               'BUCKET = "demo-shop-uploads"\n'),
        _write(dest, "app/notify.py",
               f'SLACK_WEBHOOK = "{F["slack_webhook"]}"\n'
               'SLACK_BOT_TOKEN = os.environ.get("SLACK_BOT_TOKEN")\n\n'
               "def notify(msg):\n    return {'text': msg}\n"),
    ], remove=["deploy/env.sh"])
    commit("Add payments, maps, DB config, tests and scanner config", [
        _write(dest, "app/payments.py", f'STRIPE_KEY = "{F["stripe_key"]}"\n\ndef charge(cents):\n    return cents\n'),
        _write(dest, "web/maps.js", f'const MAPS_KEY = "{F["google_key"]}";\nexport default MAPS_KEY;\n'),
        _write(dest, "app/db.py",
               f'db_password = "{F["db_password"]}"\n'
               f'api_token = "{F["inline_token"]}"  # secretscan:allow (demo token, documented)\n'
               + "\n".join(BENIGN_LINES) + "\n"),
        _write(dest, "tests/fixtures/sample_keys.txt", f"# fixture used by parser tests\n{F['fixture_token']}\n"),
        _write(dest, "docs/maps.md", f"Public demo key used in the docs: `{F['docs_google_key']}`\n"),
        _write(dest, ".secretscan.yml",
               "entropy_threshold: 3.5\n"
               "allowlist:\n"
               "  paths:\n    - \"tests/fixtures/*\"\n"
               "  fingerprints:\n"
               f"    - \"{fingerprint(F['docs_google_key'])}\"  # public docs demo key\n"),
    ])
    commit("Add deploy key", [
        _write(dest, "keys/deploy_key", f"{F['private_key_header']}\nFAKE-EXAMPLE-NOT-A-REAL-KEY-0000\n-----END OPENSSH PRIVATE KEY-----\n"),
    ])
    commit("Remove deploy key (oops)", remove=["keys/deploy_key"])
    return repo


def _supports_initial_branch() -> bool:
    try:
```

09-git-secret-scanner/src/sample_repo.py (continued)

```
        v = git.Git().version_info
        return v >= (2, 28)
    except Exception:
        return False

```

```
"""Core scanning: lines -> findings, unified diffs, git history, staged changes.

Two history modes:
  * ``patch`` (default): for every commit on every ref, diff against its first
    parent (or the empty tree for a root commit) and scan only the *added*
    lines. This attributes each secret to the commit that introduced it and
    finds secrets that were later deleted or rotated.
  * ``blobs``: scan every unique blob reachable from any commit tree (each blob
    scanned once, keyed by blob SHA). Slower, but independent of diff quirks.
"""
from __future__ import annotations

import re
import time
from dataclasses import asdict, dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from typing import Iterable, Iterator, Optional

import git

from .config import Config
from .entropy import shannon_entropy
from .redact import fingerprint, redact
from .rules import GENERIC_RULE_ID, RULES, generic_candidates

EMPTY_TREE = "4b825dc642cb6eb9a060e54bf8d69288fbee4904"  # git's well-known empty tree id
SEVERITY_ORDER = {"critical": 0, "high": 1, "medium": 2, "low": 3}


@dataclass
class Finding:
    """One occurrence of a secret. ``secret`` is held in memory only, never serialised."""
    rule_id: str
    description: str
    severity: str
    path: str
    line: int
    fingerprint: str
    redacted: str
    entropy: float
    length: int
    commit: Optional[str] = None
    commit_date: Optional[str] = None
    author: Optional[str] = None
    source: str = "patch"
    suppressed_by: Optional[str] = None
    secret: str = field(default="", repr=False)

    def public(self) -> dict:
        d = asdict(self)
        d.pop("secret")
        return d


@dataclass
class ScanResult:
    target: str
    mode: str
    findings: list[Finding] = field(default_factory=list)       # reported occurrences
    suppressed: list[Finding] = field(default_factory=list)     # allowlisted occurrences
    commits_scanned: int = 0
    blobs_scanned: int = 0
    lines_scanned: int = 0
    duration_ms: float = 0.0
```

```
    head_fingerprints: set = field(default_factory=set)

    def unique(self) -> list[dict]:
        """Dedupe occurrences by fingerprint; one row per distinct secret."""
        groups: dict[str, list[Finding]] = {}
        for f in self.findings:
            groups.setdefault(f.fingerprint, []).append(f)
        rows = []
        for fp, occ in groups.items():
            occ_sorted = sorted(occ, key=lambda f: (f.commit_date or "", f.path, f.line))
            first, last = occ_sorted[0], occ_sorted[-1]
            rows.append({
                "fingerprint": fp,
                "rule_id": first.rule_id,
                "description": first.description,
                "severity": first.severity,
                "redacted": first.redacted,
                "length": first.length,
                "entropy": round(first.entropy, 3),
                "status": "active_in_HEAD" if fp in self.head_fingerprints else "removed_from_HEAD_still_in_history",
                "occurrence_count": len(occ),
                "files": sorted({f.path for f in occ}),
                "first_seen": {"commit": first.commit, "date": first.commit_date, "path": first.path, "line": first.line},
                "last_seen": {"commit": last.commit, "date": last.commit_date, "path": last.path, "line": last.line},
                "occurrences": [
                    {"commit": f.commit, "date": f.commit_date, "path": f.path, "line": f.line, "source": f.source}
                    for f in occ_sorted
                ],
            })
        rows.sort(key=lambda r: (SEVERITY_ORDER.get(r["severity"], 9), r["first_seen"]["date"] or "", r["rule_id"]))
        return rows


# ---------------------------------------------------------------------------
# line level
# ---------------------------------------------------------------------------
def scan_line(line: str, path: str, lineno: int, cfg: Config, **meta) -> tuple[list[Finding], list[Finding]]:
    """Scan one line of text. Returns (reported, suppressed) findings."""
    if len(line) > cfg.max_line_length:
        line = line[: cfg.max_line_length]
    hits: list[tuple[str, str, str, str, int, int]] = []  # rule_id, desc, sev, secret, start, end
    for rule in RULES:
        if rule.rule_id in cfg.disabled_rules:
            continue
        if rule.keyword_hint and rule.keyword_hint.lower() not in line.lower():
            continue
        for m in rule.pattern.finditer(line):
            secret = rule.secret_from(m)
            if rule.min_entropy and shannon_entropy(secret) < rule.min_entropy:
                continue
            start = m.start(1) if m.groups() else m.start()
            hits.append((rule.rule_id, rule.description, rule.severity, secret, start, start + len(secret)))
    if GENERIC_RULE_ID not in cfg.disabled_rules:
        for secret, h, s, e in generic_candidates(line, cfg.entropy_threshold, cfg.min_generic_length):
            # skip values a provider rule already explains (overlapping span)
            if any(not (e <= hs or s >= he) for *_, hs, he in hits):
                continue
```

```
            hits.append((GENERIC_RULE_ID, "High-entropy value assigned to a credential-like name", "medium", secret, s, e))

    reported, suppressed = [], []
    inline = Config.inline_suppressed(line)
    path_ok = cfg.path_allowed(path)
    for rule_id, desc, sev, secret, _s, _e in hits:
        # A PEM header line is identical for every key, so the key's location is part of
        # its identity; otherwise all private keys in a repo would collapse into one.
        fp = fingerprint(f"{secret}@{path}" if rule_id == "private-key-block" else secret)
        f = Finding(rule_id=rule_id, description=desc, severity=sev, path=path, line=lineno,
                    fingerprint=fp, redacted=redact(secret), entropy=round(shannon_entropy(secret), 3),
                    length=len(secret), secret=secret, **meta)
        reason = None
        if inline:
            reason = "inline"
        elif path_ok:
            reason = "path"
        elif cfg.fingerprint_allowed(fp):
            reason = "fingerprint"
        elif cfg.value_allowed(secret):
            reason = "value_regex"
        if reason:
            f.suppressed_by = reason
            suppressed.append(f)
        else:
            reported.append(f)
    return reported, suppressed


def scan_text(text: str, path: str, cfg: Config, **meta) -> tuple[list[Finding], list[Finding], int]:
    rep, sup, n = [], [], 0
    for i, line in enumerate(text.splitlines(), start=1):
        n += 1
        r, s = scan_line(line, path, i, cfg, **meta)
        rep += r
        sup += s
    return rep, sup, n


# ---------------------------------------------------------------------------
# unified diff parsing
# ---------------------------------------------------------------------------
_HUNK = re.compile(r"^@@ -\d+(?:,\d+)? \+(\d+)(?:,\d+)? @@")


def added_lines(diff_text: str) -> Iterator[tuple[str, int, str]]:
    """Yield (path, new_line_number, content) for every '+' line in a unified diff (-U0 or wider)."""
    path: Optional[str] = None
    new_no = 0
    for raw in diff_text.splitlines():
        if raw.startswith("diff --git "):
            path = None
            continue
        if raw.startswith("+++ "):
            target = raw[4:].strip()
            if target.startswith('"') and target.endswith('"'):
                target = target[1:-1]
            path = None if target == "/dev/null" else (target[2:] if target.startswith("b/") else target)
            continue
        if raw.startswith("--- ") or raw.startswith("Binary files"):
```

```
            continue
        m = _HUNK.match(raw)
        if m:
            new_no = int(m.group(1))
            continue
        if path is None:
            continue
        if raw.startswith("+"):
            yield path, new_no, raw[1:]
            new_no += 1
        elif raw.startswith(" "):
            new_no += 1  # context line (only present when -U > 0)


def scan_diff(diff_text: str, cfg: Config, **meta) -> tuple[list[Finding], list[Finding], int]:
    rep, sup, n = [], [], 0
    for path, lineno, content in added_lines(diff_text):
        n += 1
        r, s = scan_line(content, path, lineno, cfg, **meta)
        rep += r
        sup += s
    return rep, sup, n


_DIFF_ARGS = ("-U0", "--no-color", "--no-ext-diff", "-M", "--src-prefix=a/", "--dst-prefix=b/")


def _is_binary(data: bytes) -> bool:
    return b"\0" in data[:8000]


def _blob_text(blob) -> Optional[str]:
    data = blob.data_stream.read()
    if _is_binary(data):
        return None
    return data.decode("utf-8", errors="replace")


def _head_fingerprints(repo: git.Repo, cfg: Config) -> set:
    """Fingerprints of every secret (reported or suppressed) present in HEAD's tree."""
    fps: set = set()
    try:
        tree = repo.head.commit.tree
    except ValueError:  # unborn HEAD
        return fps
    for item in tree.traverse():
        if item.type != "blob":
            continue
        text = _blob_text(item)
        if text is None:
            continue
        rep, sup, _ = scan_text(text, item.path, cfg)
        fps.update(f.fingerprint for f in rep + sup)
    return fps


def scan_history(repo_path: str | Path, cfg: Optional[Config] = None, mode: str = "patch",
                 rev: Optional[str] = None, max_commits: Optional[int] = None) -> ScanResult:
    """Scan every commit reachable from all refs (or from ``rev``)."""
    if mode not in ("patch", "blobs"):
        raise ValueError("mode must be 'patch' or 'blobs'")
    t0 = time.perf_counter()
    repo = git.Repo(repo_path)
    cfg = cfg or Config.load(repo_root=repo.working_tree_dir)
    result = ScanResult(target=Path(repo.working_tree_dir or repo_path).name, mode=mode)
```

```
    commits = list(repo.iter_commits(rev) if rev else repo.iter_commits("--all"))
    if max_commits:
        commits = commits[:max_commits]
    commits.reverse()  # oldest first
    seen_blobs: set = set()
    for c in commits:
        meta = dict(commit=c.hexsha[:12],
                    commit_date=datetime.fromtimestamp(c.committed_date, tz=timezone.utc).isoforma
t(),
                    author=c.author.name, source=mode)
        result.commits_scanned += 1
        if mode == "patch":
            parent = c.parents[0].hexsha if c.parents else EMPTY_TREE
            diff_text = repo.git.diff(parent, c.hexsha, *_DIFF_ARGS)
            rep, sup, n = scan_diff(diff_text, cfg, **meta)
        else:
            rep, sup, n = [], [], 0
            for item in c.tree.traverse():
                if item.type != "blob" or item.hexsha in seen_blobs:
                    continue
                seen_blobs.add(item.hexsha)
                text = _blob_text(item)
                result.blobs_scanned += 1
                if text is None:
                    continue
                r, s, k = scan_text(text, item.path, cfg, **meta)
                rep += r; sup += s; n += k
        result.findings += rep
        result.suppressed += sup
        result.lines_scanned += n
    result.head_fingerprints = _head_fingerprints(repo, cfg)
    result.duration_ms = round((time.perf_counter() - t0) * 1000, 1)
    return result


def scan_staged(repo_path: str | Path, cfg: Optional[Config] = None) -> ScanResult:
    """Scan only the lines added in the index (what ``git commit`` is about to record)."""
    t0 = time.perf_counter()
    repo = git.Repo(repo_path)
    cfg = cfg or Config.load(repo_root=repo.working_tree_dir)
    diff_text = repo.git.diff("--cached", *_DIFF_ARGS)
    rep, sup, n = scan_diff(diff_text, cfg, commit="STAGED", source="staged")
    res = ScanResult(target=Path(repo.working_tree_dir).name, mode="staged",
                     findings=rep, suppressed=sup, lines_scanned=n)
    res.head_fingerprints = {f.fingerprint for f in rep}  # staged secrets are about to become act
ive
    res.duration_ms = round((time.perf_counter() - t0) * 1000, 1)
    return res

```

09-git-secret-scanner/tests/conftest.py

```
import shutil
import subprocess
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from src.sample_repo import create_sample_repo  # noqa: E402


@pytest.fixture(scope="session")
def sample_repo(tmp_path_factory):
    dest = tmp_path_factory.mktemp("demo") / "demo_repo"
    create_sample_repo(dest)
    return dest


@pytest.fixture
def empty_repo(tmp_path):
    """A fresh git repo with an identity configured (needed for `git commit`)."""
    d = tmp_path / "repo"
    d.mkdir()
    for args in (["init", "-q"], ["config", "user.email", "t@example.com"], ["config", "user.name", "Test"],
                 ["config", "commit.gpgsign", "false"]):
        subprocess.run(["git", *args], cwd=d, check=True)
    return d


def git(repo, *args, check=True):
    return subprocess.run(["git", *args], cwd=repo, capture_output=True, text=True, check=check)

```

09-git-secret-scanner/tests/test_app_selfscan.py

```
"""The Streamlit app runs offline without exceptions, and this project's own files contain no
secret-shaped strings (planted values are assembled at runtime from fragments)."""
from pathlib import Path

import pytest

from src.config import Config
from src.scanner import scan_text
from tests.conftest import ROOT


def test_project_source_self_scan_is_clean():
    hits = []
    for p in ROOT.rglob("*"):
        if not p.is_file() or any(part in {".git", "__pycache__", ".pytest_cache"} for part in p.p
arts):
            continue
        if p.suffix not in {".py", ".md", ".txt", ".json", ".html", ".yml", ".yaml", ".ini", ".sh"
}:
            continue
        rep, _sup, _n = scan_text(p.read_text(errors="replace"), str(p.relative_to(ROOT)), Config(
))
        hits += [(f.path, f.line, f.rule_id) for f in rep]
    assert hits == []


def test_streamlit_app_runs_without_exceptions():
    if not (ROOT / "outputs" / "report.json").exists():
        pytest.skip("run scripts/run_demo.py first to create outputs/report.json")
    from streamlit.testing.v1 import AppTest
    at = AppTest.from_file(str(ROOT / "app.py"), default_timeout=120).run()
    assert not at.exception
    assert len(at.tabs) == 5
    assert any(m.label == "Unique secrets" for m in at.metric)

```

```python
"""History scanning on the generated sample repo: deleted/rotated secrets, dedupe, allowlist, repo
rts."""
import json

import git
import pytest

from src.config import Config
from src.report import build_report, write_html, write_json
from src.redact import fingerprint
from src.sample_repo import EXPECTED, FAKE, create_sample_repo
from src.scanner import added_lines, scan_history
from tests.conftest import git as run_git

def fp_of(label):
    if label == "private_key_header":
        return fingerprint(FAKE[label] + "@keys/deploy_key")
    return fingerprint(FAKE[label])


REPORTED = {k: v for k, v in EXPECTED.items() if v[1] == "reported"}
SUPPRESSED = {k: v for k, v in EXPECTED.items() if v[1] != "reported"}


@pytest.fixture(scope="module")
def patch_result(sample_repo):
    return scan_history(sample_repo, mode="patch")


def test_sample_repo_is_deterministic(tmp_path):
    a = create_sample_repo(tmp_path / "a")
    b = create_sample_repo(tmp_path / "b")
    assert a.head.commit.hexsha == b.head.commit.hexsha
    assert len(list(a.iter_commits())) == 8


def test_sample_repo_refuses_overwrite(tmp_path):
    create_sample_repo(tmp_path / "a")
    with pytest.raises(FileExistsError):
        create_sample_repo(tmp_path / "a")


@pytest.mark.parametrize("mode", ["patch", "blobs"])
def test_finds_every_planted_secret_exactly(sample_repo, mode):
    res = scan_history(sample_repo, mode=mode)
    found = {u["fingerprint"]: u for u in res.unique()}
    expected = {fp_of(k): k for k in REPORTED}
    assert set(found) == set(expected)  # recall 100%, zero false positives on the sample
    for fp, label in expected.items():
        assert found[fp]["rule_id"] == REPORTED[label][0]


def test_deleted_and_rotated_secrets_marked_removed(patch_result):
    status = {u["fingerprint"]: u["status"] for u in patch_result.unique()}
    for label, (_rule, _o, st) in REPORTED.items():
        want = "active_in_HEAD" if st == "active" else "removed_from_HEAD_still_in_history"
        assert status[fp_of(label)] == want, label


def test_rotated_key_both_versions_reported_with_commits(patch_result):
    u = {r["fingerprint"]: r for r in patch_result.unique()}
    old, new = u[fingerprint(FAKE["aws_key_1"])], u[fingerprint(FAKE["aws_key_2"])]
    assert old["first_seen"]["commit"] != new["first_seen"]["commit"]
    assert old["first_seen"]["date"] < new["first_seen"]["date"]

```

```
65
66 def test_dedupe_by_fingerprint(patch_result):
67     rows = patch_result.unique()
68     aws1 = next(r for r in rows if r["fingerprint"] == fingerprint(FAKE["aws_key_1"]))
69     assert aws1["occurrence_count"] == 2
70     assert aws1["files"] == ["app/settings.py", "deploy/env.sh"]
71     assert len(patch_result.findings) == len(rows) + 1  # one duplicate collapsed
72
73
74 def test_allowlist_inline_path_fingerprint(patch_result):
75     reasons = {f.fingerprint: f.suppressed_by for f in patch_result.suppressed}
76     for label, (_rule, reason, _s) in SUPPRESSED.items():
77         assert reasons[fingerprint(FAKE[label])] == reason, label
78     reported_fps = {f.fingerprint for f in patch_result.findings}
79     assert not reported_fps & set(reasons)
80
81
82 def test_without_repo_config_path_and_fingerprint_allowlist_off(sample_repo):
83     res = scan_history(sample_repo, Config(), mode="patch")
84     fps = {f.fingerprint for f in res.findings}
85     assert fingerprint(FAKE["fixture_token"]) in fps
86     assert fingerprint(FAKE["docs_google_key"]) in fps
87     assert fingerprint(FAKE["inline_token"]) not in fps  # inline marker still honoured
88
89
90 def test_line_numbers_point_at_new_file(patch_result):
91     occ = {(f.path, f.line) for f in patch_result.findings}
92     assert ("app/payments.py", 1) in occ and ("scripts/ci.sh", 3) in occ
93
94
95 def test_added_lines_parser():
96     diff = ("diff --git a/x.py b/x.py\n--- a/x.py\n+++ b/x.py\n@@ -1,0 +5,2 @@\n+one\n+two\n"
97             "diff --git a/gone.py b/gone.py\n--- a/gone.py\n+++ /dev/null\n@@ -1 +0,0 @@\n-bye\n")
98     assert list(added_lines(diff)) == [("x.py", 5, "one"), ("x.py", 6, "two")]
99
100
101 def test_reports_contain_no_raw_secret(patch_result, tmp_path):
102     report = build_report(patch_result)
103     j = write_json(report, tmp_path / "r.json").read_text()
104     h = write_html(report, tmp_path / "r.html").read_text()
105     json.loads(j)
106     for label, value in FAKE.items():
107         if label == "private_key_header":
108             continue  # a PEM header is a marker, not a secret; still redacted below
109         for blob in (j, h):
110             assert value not in blob, label
111             assert value[4:14] not in blob, label  # no meaningful substring either
112     assert FAKE["private_key_header"] not in j and FAKE["private_key_header"] not in h
113     assert report["stats"]["unique_secrets"] == len(REPORTED)
114     assert "<table>" in h and "AKIA********(20 chars)" in h
115
116
117 def test_html_escapes_untrusted_paths(tmp_path):
118     from src.scanner import ScanResult, scan_line
119     rep, _ = scan_line(FAKE["github_token"], "<script>alert(1)</script>.txt", 1, Config(), commit=
    "abc", commit_date="2026")
120     res = ScanResult(target="t", mode="patch", findings=rep)
121     h = write_html(build_report(res), tmp_path / "x.html").read_text()
122     assert "<script>alert(1)" not in h and "&lt;script&gt;" in h
123
124
125 def test_planted_private_key_body_is_fake(sample_repo):
126     repo = git.Repo(sample_repo)
127     added = next(c for c in repo.iter_commits() if c.summary == "Add deploy key")
128     body = run_git(sample_repo, "show", f"{added.hexsha}:keys/deploy_key").stdout
```

```
    assert "FAKE-EXAMPLE-NOT-A-REAL-KEY" in body and len(body) < 200   # no real key material


def test_private_keys_in_different_files_are_distinct(tmp_path):
    from src.scanner import scan_line
    a, _ = scan_line(FAKE["private_key_header"], "a.pem", 1, Config())
    b, _ = scan_line(FAKE["private_key_header"], "b.pem", 1, Config())
    assert a[0].fingerprint != b[0].fingerprint

```

```python
"""Pre-commit hook (real `git commit` runs), staged scanning and CLI exit codes."""
import json
import subprocess
import sys
from pathlib import Path

import pytest

from src.hook import MARKER, hook_path, install_hook, uninstall_hook
from src.sample_repo import FAKE
from src.scanner import scan_staged
from tests.conftest import ROOT, git

CLI = [sys.executable, str(ROOT / "cli.py")]


def run_cli(*args, cwd=ROOT):
    return subprocess.run([*CLI, *args], cwd=cwd, capture_output=True, text=True, timeout=120)


def head_count(repo):
    r = git(repo, "rev-list", "--count", "HEAD", check=False)
    return int(r.stdout.strip()) if r.returncode == 0 else 0


def test_staged_scan_finds_only_staged_lines(empty_repo):
    (empty_repo / "ok.py").write_text("x = 1\n")
    git(empty_repo, "add", "ok.py")
    assert scan_staged(empty_repo).findings == []        # unborn HEAD works
    (empty_repo / "cfg.py").write_text(f'KEY = "{FAKE["stripe_key"]}"\n')
    assert scan_staged(empty_repo).findings == []        # not staged yet
    git(empty_repo, "add", "cfg.py")
    res = scan_staged(empty_repo)
    assert [(f.path, f.line, f.rule_id) for f in res.findings] == [("cfg.py", 1, "stripe-secret-key")]


def test_hook_blocks_commit_with_secret_and_allows_clean(empty_repo):
    install_hook(empty_repo)
    (empty_repo / "clean.py").write_text("print('hi')\n")
    git(empty_repo, "add", "clean.py")
    ok = git(empty_repo, "commit", "-q", "-m", "clean", check=False)
    assert ok.returncode == 0 and head_count(empty_repo) == 1

    (empty_repo / "leak.py").write_text(f'TOKEN = "{FAKE["github_token"]}"\n')
    git(empty_repo, "add", "leak.py")
    blocked = git(empty_repo, "commit", "-q", "-m", "leak", check=False)
    assert blocked.returncode != 0
    assert "BLOCKED" in blocked.stderr and "github-token" in blocked.stderr
    assert FAKE["github_token"] not in blocked.stderr + blocked.stdout  # hook output is redacted
    assert head_count(empty_repo) == 1                    # commit did not happen


def test_hook_respects_inline_suppression(empty_repo):
    install_hook(empty_repo)
    (empty_repo / "t.py").write_text(f'TOKEN = "{FAKE["github_token"]}"  # secretscan:allow test fixture\n')
    git(empty_repo, "add", "t.py")
    assert git(empty_repo, "commit", "-q", "-m", "fixture", check=False).returncode == 0


def test_precommit_exit_codes_direct(empty_repo):
    (empty_repo / "a.py").write_text("a = 1\n")
    git(empty_repo, "add", "a.py")
    assert run_cli("precommit", "--repo", str(empty_repo)).returncode == 0
```

```
    (empty_repo / "b.py").write_text(f'slack = "{FAKE["slack_token"]}"\n')
    git(empty_repo, "add", "b.py")
    assert run_cli("precommit", "--repo", str(empty_repo)).returncode == 1


def test_install_refuses_foreign_hook_without_force(empty_repo):
    p = hook_path(empty_repo)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text("#!/bin/sh\necho mine\n")
    with pytest.raises(FileExistsError):
        install_hook(empty_repo)
    install_hook(empty_repo, force=True)
    assert MARKER in p.read_text()
    assert p.with_name("pre-commit.secretscan.bak").read_text().endswith("echo mine\n")
    assert uninstall_hook(empty_repo) is True
    assert p.read_text().endswith("echo mine\n")          # original restored


def test_cli_install_hook_and_help(empty_repo):
    assert run_cli("--help").returncode == 0
    r = run_cli("install-hook", str(empty_repo))
    assert r.returncode == 0 and hook_path(empty_repo).exists()
    assert hook_path(empty_repo).stat().st_mode & 0o111


def test_cli_generate_and_scan(tmp_path):
    dest = tmp_path / "demo"
    assert run_cli("generate-sample", str(dest)).returncode == 0
    assert run_cli("generate-sample", str(dest)).returncode == 2          # refuses overwrite
    r = run_cli("scan", str(dest), "--json", str(tmp_path / "r.json"), "--html", str(tmp_path / "r
.html"),
                "--fail-on-findings")
    assert r.returncode == 1
    data = json.loads((tmp_path / "r.json").read_text())
    assert data["stats"]["unique_secrets"] == 11 and data["stats"]["suppressed_occurrences"] == 3
    for v in FAKE.values():
        assert v not in r.stdout


def test_cli_scan_clean_repo_exit_zero(empty_repo):
    (empty_repo / "a.py").write_text("print(1)\n")
    git(empty_repo, "add", "a.py")
    git(empty_repo, "commit", "-q", "-m", "a")
    assert run_cli("scan", str(empty_repo), "--fail-on-findings").returncode == 0


def test_cli_scan_does_not_modify_repo(sample_repo):
    before = git(sample_repo, "status", "--porcelain").stdout
    head = git(sample_repo, "rev-parse", "HEAD").stdout
    run_cli("scan", str(sample_repo))
    assert git(sample_repo, "status", "--porcelain").stdout == before
    assert git(sample_repo, "rev-parse", "HEAD").stdout == head

```

```python
"""Unit tests for rules, entropy, redaction and fingerprinting."""
import math

import pytest

from src.config import Config
from src.entropy import shannon_entropy
from src.redact import fingerprint, redact
from src.sample_repo import BENIGN_LINES, EXPECTED, FAKE
from src.scanner import scan_line


def test_entropy_known_values():
    assert shannon_entropy("") == 0.0
    assert shannon_entropy("aaaa") == 0.0
    assert math.isclose(shannon_entropy("ab"), 1.0)
    assert math.isclose(shannon_entropy("abcdefgh"), 3.0)


@pytest.mark.parametrize("label", [k for k, v in EXPECTED.items() if v[1] != "inline"])
def test_each_fake_secret_matches_its_rule(label):
    rule_id = EXPECTED[label][0]
    value = FAKE[label]
    if rule_id.startswith("generic"):
        line = f'my_secret_token = "{value}"'
    elif rule_id == "aws-secret-access-key":  # bare 40-char base64 is too generic: needs AWS cont
ext
        line = f"aws_secret_access_key: {value}"
    else:
        line = f"value: {value}"
    rep, sup = scan_line(line, "x.txt", 1, Config())
    assert [f.rule_id for f in rep] == [rule_id], (label, [f.rule_id for f in rep])
    assert rep[0].secret == value


@pytest.mark.parametrize("line", BENIGN_LINES + [
    "import hashlib  # sha256 of nothing",
    "AKIA_PREFIX = 'AKIA'",                 # too short to be a key id
    "url = 'https://hooks.slack.com/'",     # incomplete webhook
    "ghp_short = 'ghp_abc'",
])
def test_benign_lines_not_reported(line):
    rep, sup = scan_line(line, "app.py", 1, Config())
    assert rep == [] and sup == []


def test_generic_rule_respects_entropy_threshold():
    line = 'client_secret = "Ab3$Cd5%Ef7&Gh9*Jk"'  # secretscan:allow (synthetic test value)
    line = line.split("  #")[0]
    assert scan_line(line, "a.py", 1, Config(entropy_threshold=3.5))[0]
    assert not scan_line(line, "a.py", 1, Config(entropy_threshold=5.0))[0]


def test_provider_match_not_double_counted_by_generic_rule():
    rep, _ = scan_line(f'AWS_SECRET_ACCESS_KEY = "{FAKE["aws_secret_1"]}"', "s.py", 3, Config())
    assert [f.rule_id for f in rep] == ["aws-secret-access-key"]


def test_disabled_rule():
    rep, _ = scan_line(FAKE["github_token"], "a", 1, Config(disabled_rules=["github-token"]))
    assert rep == []


def test_fingerprint_stable_and_not_reversible_text():
    fp = fingerprint(FAKE["aws_key_1"])
```

```
    assert fp == fingerprint(FAKE["aws_key_1"]) and len(fp) == 24
    assert fp != fingerprint(FAKE["aws_key_2"])
    assert FAKE["aws_key_1"] not in fp


def test_redaction_reveals_only_public_prefix():
    assert redact(FAKE["aws_key_1"]) == "AKIA********(20 chars)"
    r = redact(FAKE["db_password"])  # generic secret: no characters at all
    assert r == "********(27 chars)"
    for label, v in FAKE.items():
        assert v not in redact(v)
        assert v[4:10] not in redact(v)


def test_all_planted_secrets_are_visibly_fake():
    """Every planted value carries FAKE/EXAMPLE/TEST or is a vendor-documented all-zero/X example.
"""
    for label, v in FAKE.items():
        if label == "private_key_header":
            continue  # the PEM header is a generic marker; the key body is checked in test_histor
y
        u = v.upper()
        assert ("FAKE" in u or "EXAMPLE" in u or "TEST" in u or "0" * 8 in v or "X" * 20 in v), la
bel


def test_fake_secret_shapes():
    assert len(FAKE["aws_key_1"]) == 20 and len(FAKE["aws_key_2"]) == 20
    assert len(FAKE["aws_secret_1"]) == 40 and len(FAKE["aws_secret_2"]) == 40
    assert len(FAKE["github_token"]) == 40 and len(FAKE["google_key"]) == 39


def test_aws_secret_requires_aws_context():
    assert scan_line(f"value: {FAKE['aws_secret_1']}", "x", 1, Config())[0] == []

```

09-git-secret-scanner/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.db
# generated demo repositories (contain FAKE planted secrets + a .git folder) - regenerate with cli.py generate-sample
sample_repo/
demo_repo/
*.secretscan.bak

```

09-git-secret-scanner/config/secretscan.example.yml

```
# Copy to <your repo>/.secretscan.yml (picked up automatically) or pass --config.
entropy_threshold: 3.5      # bits/char a generic credential value must reach to be reported
min_generic_length: 16      # shorter quoted values are ignored by the generic rule
disabled_rules: []          # e.g. [generic-high-entropy-assignment]
allowlist:
  paths:                    # fnmatch globs on repo-relative paths
    - "tests/fixtures/*"
    - "docs/examples/*"
  fingerprints: []          # copy the 24-hex fingerprint of a reviewed false positive from a report
  values_regex: []          # regexes matched against the candidate value; use sparingly

```

09-git-secret-scanner/pytest.ini

```
[pytest]
pythonpath = .
testpaths = tests
timeout = 120
markers =
    network: needs internet access

```

```python
"""End-to-end demo used to produce outputs/: generate the FAKE sample repo in a temp dir,
scan its history in both modes, write JSON/HTML reports + a chart, and exercise the
pre-commit hook with real `git commit` calls. The temp repo (with its .git) is deleted.

Run:  python scripts/run_demo.py
"""
from __future__ import annotations

import json
import shutil
import subprocess
import sys
import tempfile
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

import matplotlib  # noqa: E402
matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402

from src.hook import install_hook  # noqa: E402
from src.report import build_report, format_console, write_html, write_json  # noqa: E402
from src.sample_repo import EXPECTED, FAKE, create_sample_repo  # noqa: E402
from src.scanner import scan_history  # noqa: E402

OUT = ROOT / "outputs"


def sh(cmd, cwd):
    return subprocess.run(cmd, cwd=cwd, capture_output=True, text=True, timeout=120)


def main() -> dict:
    OUT.mkdir(exist_ok=True)
    log: list[str] = []
    tmp = Path(tempfile.mkdtemp(prefix="secretscan-demo-"))
    try:
        repo_dir = tmp / "demo_repo"
        repo = create_sample_repo(repo_dir)
        commits = list(repo.iter_commits("--all"))
        log.append(f"$ python cli.py generate-sample demo_repo\ncreated demo repo with {len(commit
s)} commits (all planted secrets are FAKE examples)")
        for c in reversed(commits):
            log.append(f"  {c.hexsha[:12]}  {c.summary}")

        res = scan_history(repo_dir, mode="patch")
        res.target = "demo_repo"
        report = build_report(res)
        write_json(report, OUT / "report.json")
        write_html(report, OUT / "report.html")
        log.append("\n$ python cli.py scan demo_repo --json outputs/report.json --html outputs/rep
ort.html")
        log.append(format_console(res))

        bres = scan_history(repo_dir, mode="blobs")
        bres.target = "demo_repo"
        breport = build_report(bres)
        write_json(breport, OUT / "report_blobs.json")
        log.append("\n$ python cli.py scan demo_repo --mode blobs")
        log.append(format_console(bres).splitlines()[0])

        # ground-truth evaluation
```

```
        n_expected_reported = sum(1 for v in EXPECTED.values() if v[1] == "reported")
        n_expected_suppressed = sum(1 for v in EXPECTED.values() if v[1] != "reported")

        # pre-commit hook demo (real git commits)
        hook_repo = tmp / "hook_repo"
        hook_repo.mkdir()
        for a in (["init", "-q"], ["config", "user.email", "demo@example.com"], ["config", "user.name", "Demo"],
                  ["config", "commit.gpgsign", "false"]):
            sh(["git", *a], hook_repo)
        install_hook(hook_repo)
        log.append("\n$ python cli.py install-hook hook_repo\ninstalled pre-commit hook at hook_repo/.git/hooks/pre-commit")
        (hook_repo / "app.py").write_text("print('hello')\n")
        sh(["git", "add", "app.py"], hook_repo)
        t0 = time.perf_counter()
        ok = sh(["git", "commit", "-m", "clean commit"], hook_repo)
        t_clean = (time.perf_counter() - t0) * 1000
        log.append(f"\n$ git commit -m 'clean commit'   -> exit {ok.returncode}\n{ok.stdout.strip()}")
        (hook_repo / "config.py").write_text(f'STRIPE_KEY = "{FAKE["stripe_key"]}"\n')
        sh(["git", "add", "config.py"], hook_repo)
        t0 = time.perf_counter()
        bad = sh(["git", "commit", "-m", "add stripe key"], hook_repo)
        t_block = (time.perf_counter() - t0) * 1000
        log.append(f"\n$ git commit -m 'add stripe key'   -> exit {bad.returncode}\n{bad.stderr.strip()}")
        n_commits_after = int(sh(["git", "rev-list", "--count", "HEAD"], hook_repo).stdout.strip())

        # chart: unique secrets by rule, coloured by status
        rows = report["secrets"]
        rules = sorted({r["rule_id"] for r in rows})
        active = [sum(1 for r in rows if r["rule_id"] == k and r["status"] == "active_in_HEAD") for k in rules]
        removed = [sum(1 for r in rows if r["rule_id"] == k and r["status"] != "active_in_HEAD") for k in rules]
        fig, ax = plt.subplots(figsize=(9, 4.5))
        ax.barh(rules, active, color="#b42318", label="still in HEAD")
        ax.barh(rules, removed, left=active, color="#98a2b3", label="removed from HEAD (still in history)")
        ax.set_xlabel("unique secrets (deduplicated by fingerprint)")
        ax.set_title(f"Demo repo: {len(rows)} unique FAKE secrets across {res.commits_scanned} commits")
        ax.legend(loc="upper center", bbox_to_anchor=(0.5, -0.15), ncol=2, frameon=False)
        ax.xaxis.get_major_locator().set_params(integer=True)
        fig.tight_layout()
        fig.savefig(OUT / "findings_by_rule.png", dpi=120)
        plt.close(fig)

        summary = {
            "commits": res.commits_scanned,
            "lines_scanned_patch": res.lines_scanned,
            "raw_occurrences_patch": len(res.findings),
            "unique_secrets": len(rows),
            "expected_unique": n_expected_reported,
            "suppressed": len(res.suppressed),
            "expected_suppressed": n_expected_suppressed,
            "active_in_head": report["stats"]["active_in_head"],
            "removed_but_in_history": report["stats"]["removed_but_in_history"],
            "raw_occurrences_blobs": len(bres.findings),
            "unique_secrets_blobs": len(breport["secrets"]),
            "blobs_scanned": bres.blobs_scanned,
            "scan_ms_patch": res.duration_ms,
            "scan_ms_blobs": bres.duration_ms,
```

```
            "hook_clean_exit": ok.returncode,
            "hook_secret_exit": bad.returncode,
            "hook_commits_recorded": n_commits_after,
            "hook_ms_clean": round(t_clean, 1),
            "hook_ms_block": round(t_block, 1),
        }
        # leak self-check over every artifact written
        blobs = "".join((OUT / f).read_text() for f in ("report.json", "report.html", "report_blobs.json")) + "\n".join(log)
        leaked = [k for k, v in FAKE.items() if k != "private_key_header" and v in blobs]
        summary["raw_secrets_in_outputs"] = len(leaked)
        log.append(f"\nself-check: raw planted secrets found in report.json/report.html/report_blobs.json/console output: {len(leaked)}")
        (OUT / "sample_output.txt").write_text("\n".join(log) + "\n")
        (OUT / "demo_summary.json").write_text(json.dumps(summary, indent=2))
        print("\n".join(log))
        print(json.dumps(summary, indent=2))
        return summary
    finally:
        shutil.rmtree(tmp, ignore_errors=True)


if __name__ == "__main__":
    main()

```

```
"""Measure false positives / runtime on real public repositories (network required).

Clones each repo into a temp dir (full history), scans every commit in patch mode with
the default config, and writes outputs/public_repo_scans.json (redacted, no raw values).
The clones are deleted afterwards.

Run:  python scripts/scan_public_repos.py
"""
from __future__ import annotations

import json
import shutil
import subprocess
import sys
import tempfile
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))

from src.config import Config  # noqa: E402
from src.scanner import scan_history  # noqa: E402

REPOS = [
    "https://github.com/pypa/sampleproject",
    "https://github.com/pallets/itsdangerous",
    "https://github.com/psf/requests",
    "https://github.com/boto/boto3",
]


def main(repos=REPOS, workdir: str | None = None) -> list[dict]:
    tmp = Path(workdir or tempfile.mkdtemp(prefix="secretscan-public-"))
    results = []
    try:
        for url in repos:
            name = url.rstrip("/").split("/")[-1]
            dest = tmp / name
            if not dest.exists():
                subprocess.run(["git", "clone", "-q", url, str(dest)], check=True, timeout=600)
            head = subprocess.run(["git", "log", "-1", "--format=%H %cs"], cwd=dest, capture_outpu
t=True,
                                  text=True).stdout.split()
            res = scan_history(dest, Config(), mode="patch")
            uniq = res.unique()
            results.append({
                "repo": url, "head_commit": head[0], "head_date": head[1],
                "commits_scanned": res.commits_scanned, "added_lines_scanned": res.lines_scanned,
                "raw_occurrences": len(res.findings), "unique_findings": len(uniq),
                "duration_s": round(res.duration_ms / 1000, 1),
                "findings": [{"rule_id": u["rule_id"], "redacted": u["redacted"], "files": u["file
s"][:5],
                              "status": u["status"], "occurrences": u["occurrence_count"]} for u i
n uniq],
            })
            print(json.dumps(results[-1])[:600])
    finally:
        if workdir is None:
            shutil.rmtree(tmp, ignore_errors=True)
    (ROOT / "outputs").mkdir(exist_ok=True)
    (ROOT / "outputs" / "public_repo_scans.json").write_text(json.dumps(results, indent=2))
    return results


if __name__ == "__main__":
```

09-git-secret-scanner/scripts/scan_public_repos.py (continued)

```
63      main()
64
```